



廈門大學

# 《计算机组成原理》 课程实验报告

姓名：马鑫

学院：信息学院

专业：数字媒体技术

学号：37220232203780

2026 年 4 月

## 第 4 次实验 指令系统

### 1. 实验目的

- (1) 掌握 MIPS 汇编仿真器 (MARS 4.5)、RISC-V 汇编仿真器 (RARS 1.5)、Intel x86 汇编工具 (masm32)、ARMv7 在线汇编工具的使用方法。
- (2) 在 Logisim 实现的单周期 MIPS 处理器、Logisim 实现的单周期 RISC-V 处理器上运行程序。
- (3) 分别使用 MIPS 指令、RISC-V 指令、x86 指令、ARMv7 指令编写排序程序。

### 2. 实验环境

- (1) MARS 4.5、RARS 1.5、masm32、ARMv7 在线汇编工具。
- (2) Windows 系统下运行 Logisim 软件 (需安装 JDK)。

### 3. 实验内容

#### 3.1 验证实验

##### (1) MIPS 汇编语言程序的运行

##### a. 运行 MIPS 汇编仿真器

电脑上已有 JDK。

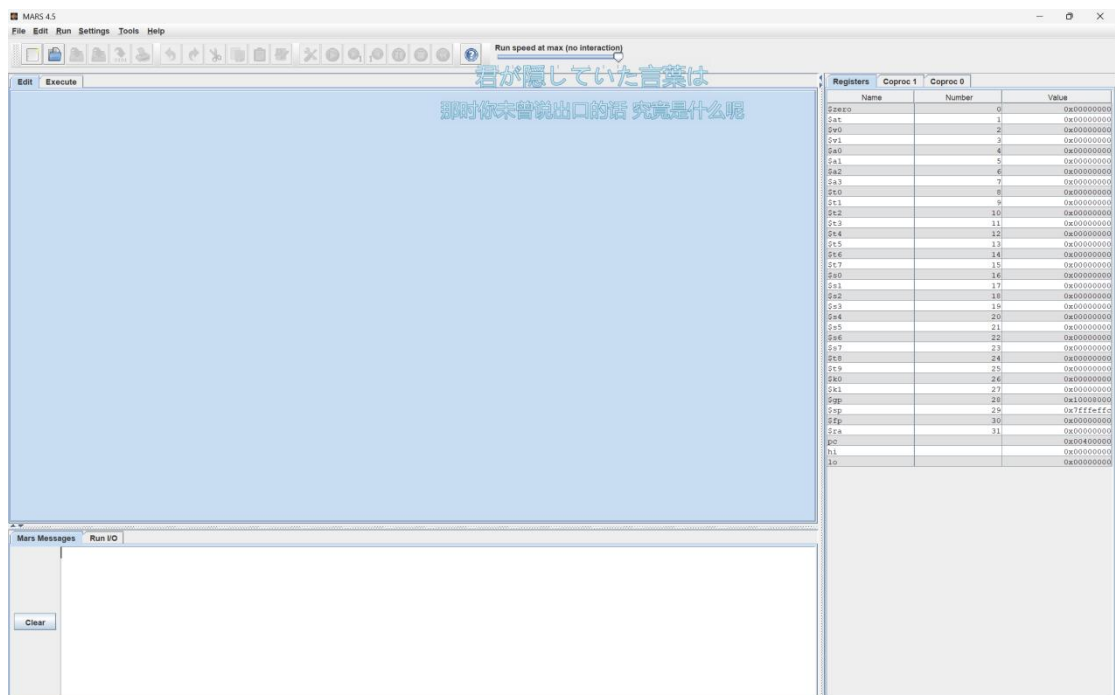
```
C:\Users\koshitann>java -version
java version "21.0.9" 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 21.0.9+7-LTS-338)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.9+7-LTS-338, mixed mode, sharing)
```

运行 MIPS 汇编仿真器 Mars4\_5.jar。

```
C:\Users\koshitann>cd E:\lab4\mips

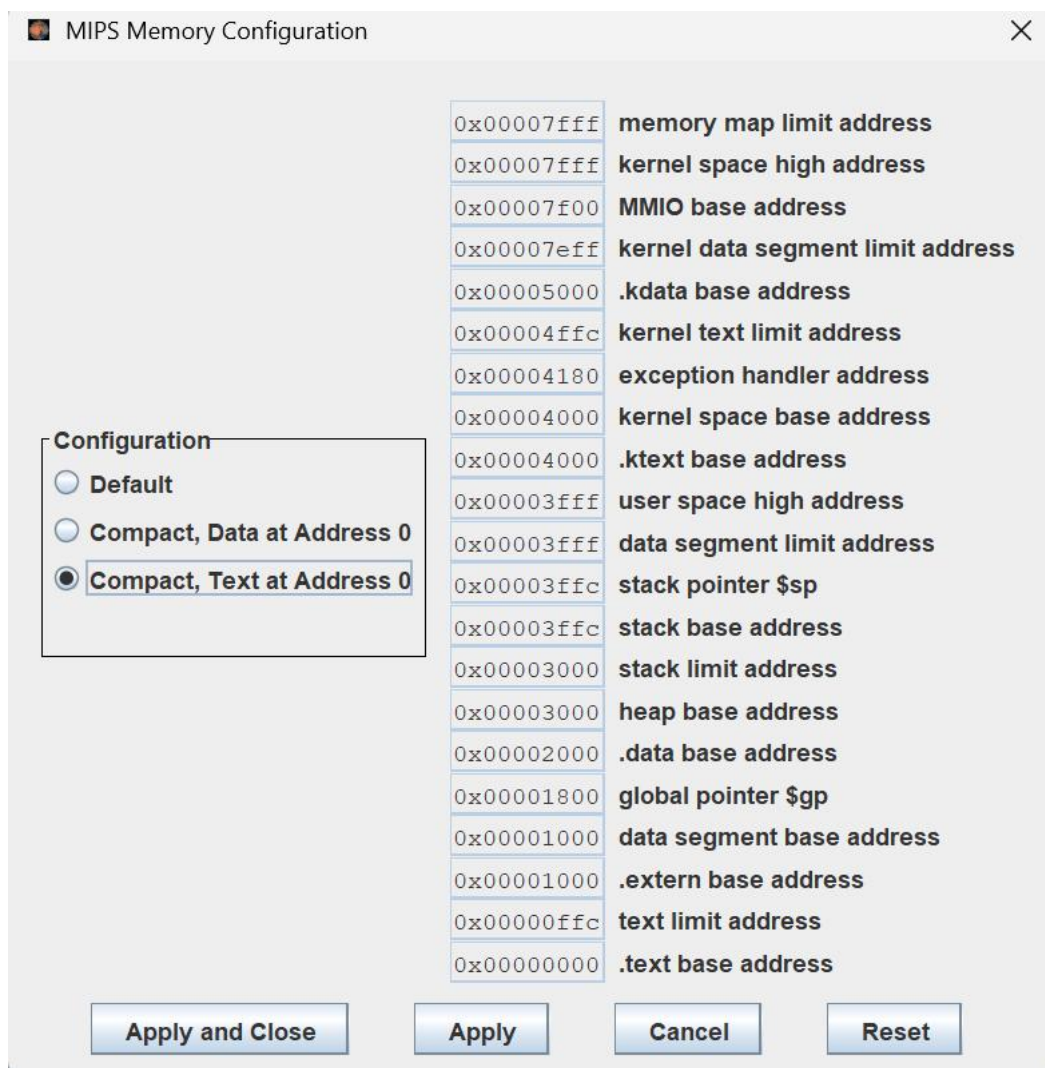
C:\Users\koshitann>E:

E:\lab4\mips>java -jar Mars4_5.jar
|
```



b. 运行第一个 MIPS 汇编语言程序

第一步：将 MARS 汇编仿真器的代码段的开始地址设置为 0



第二步：在 MARS 汇编仿真器中打开 helloworld\_mips.asm

```

helloworld_mips.asm
1  #MIPS程序      helloworld_mips.asm      这个程序只能在MARS汇编器上运行
2
3  .data          #数据段开始标志
4      out_string: .asciiz "\nHello, World!\n"
5
6  .text          #代码段开始标志
7  main:
8      li $v0,4      #4号系统调用，显示字符串
9      la $a0,out_string #获取字符串的地址
10     syscall        #系统调用
11
12     li $v0,10      #10号系统调用，程序退出
13     syscall
14

```

第三步：对 helloworld\_mips.asm 源程序进行汇编



| Edit Execute             |            |            |                          |        |                              |
|--------------------------|------------|------------|--------------------------|--------|------------------------------|
| Text Segment             |            |            |                          |        |                              |
| Bkpt                     | Address    | Code       | Basic                    | Source |                              |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000.. | 8:     | li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000  | 9:     | la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                  | 10:    | syscall #系统调用                |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000.. | 12:    | li \$v0,10 #10号系统调用，程序退出     |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                  | 13:    | syscall                      |

第四步： 运行汇编后的程序，在下面的界面中显示“Hello, World!”

Mars Messages Run I/O

Hello, World!  
  
-- program is finished running --

Clear

Mars Messages Run I/O

Assemble: assembling E:\lab4\mips\helloworld\_mips.asm  
  
Assemble: operation completed successfully.  
  
Go: running helloworld\_mips.asm  
  
Go: execution completed successfully.

Clear

第五步：也可以单步执行程序：首先点击： Reset，此时将第一条指令背景置黄色，表示从第一条指令开始执行；然后点击： Step，每点击一次 Step，执行一条指令；直到所有指令执行完毕。

| Edit Execute             |            |            |                          |        |                              |
|--------------------------|------------|------------|--------------------------|--------|------------------------------|
| Text Segment             |            |            |                          |        |                              |
| Bkpt                     | Address    | Code       | Basic                    | Source |                              |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000.. | 8:     | li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000  | 9:     | la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                  | 10:    | syscall #系统调用                |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000.. | 12:    | li \$v0,10 #10号系统调用，程序退出     |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                  | 13:    | syscall                      |

| Edit Execute             |            |            |                          |        |                              |
|--------------------------|------------|------------|--------------------------|--------|------------------------------|
| Text Segment             |            |            |                          |        |                              |
| Bkpt                     | Address    | Code       | Basic                    | Source |                              |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000.. | 8:     | li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000  | 9:     | la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                  | 10:    | syscall #系统调用                |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000.. | 12:    | li \$v0,10 #10号系统调用，程序退出     |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                  | 13:    | syscall                      |

| Edit Execute             |            |            |                           |                                 |
|--------------------------|------------|------------|---------------------------|---------------------------------|
| Text Segment             |            |            |                           |                                 |
| Bkpt                     | Address    | Code       | Basic                     | Source                          |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000... | 8: li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000   | 9: la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                   | 10: syscall #系统调用               |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000... | 12: li \$v0,10 #10号系统调用，程序退出    |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                   | 13: syscall                     |

| Edit Execute             |            |            |                           |                                 |
|--------------------------|------------|------------|---------------------------|---------------------------------|
| Text Segment             |            |            |                           |                                 |
| Bkpt                     | Address    | Code       | Basic                     | Source                          |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000... | 8: li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000   | 9: la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                   | 10: syscall #系统调用               |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000... | 12: li \$v0,10 #10号系统调用，程序退出    |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                   | 13: syscall                     |

| Text Segment             |            |            |                           |                                 |
|--------------------------|------------|------------|---------------------------|---------------------------------|
| Bkpt                     | Address    | Code       | Basic                     | Source                          |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000... | 8: li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000   | 9: la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                   | 10: syscall #系统调用               |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000... | 12: li \$v0,10 #10号系统调用，程序退出    |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                   | 13: syscall                     |

| Edit Execute             |            |            |                           |                                 |
|--------------------------|------------|------------|---------------------------|---------------------------------|
| Text Segment             |            |            |                           |                                 |
| Bkpt                     | Address    | Code       | Basic                     | Source                          |
| <input type="checkbox"/> | 0x00000000 | 0x24020004 | addiu \$2,\$0,0x000000... | 8: li \$v0,4 #4号系统调用，显示字符串      |
| <input type="checkbox"/> | 0x00000004 | 0x20042000 | addi \$4,\$0,0x00002000   | 9: la \$a0,out_string #获取字符串的地址 |
| <input type="checkbox"/> | 0x00000008 | 0x0000000c | syscall                   | 10: syscall #系统调用               |
| <input type="checkbox"/> | 0x0000000c | 0x2402000a | addiu \$2,\$0,0x000000... | 12: li \$v0,10 #10号系统调用，程序退出    |
| <input type="checkbox"/> | 0x00000010 | 0x0000000c | syscall                   | 13: syscall                     |

| Mars Messages                                                                                                                                           |  | Run I/O |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|--|---------|
| <div> <div>Clear</div> <pre> -- program is finished running --  Reset: reset completed.  Hello, World!  -- program is finished running -- </pre> </div> |  |         |

c. 求累加和的 MIPS 汇编语言程序

第一步：将 MARS 汇编仿真器的数据段的开始地址设置为 0

Configuration

☐ Default
 ☒ Compact, Data at Address 0
 ☐ Compact, Text at Address 0

|            |                                   |
|------------|-----------------------------------|
| 0x00007fff | memory map limit address          |
| 0x00007fff | kernel space high address         |
| 0x00007f00 | MMIO base address                 |
| 0x00007eff | kernel data segment limit address |
| 0x00005000 | .kdata base address               |
| 0x00004ffc | kernel text limit address         |
| 0x00004180 | exception handler address         |
| 0x00004000 | kernel space base address         |
| 0x00004000 | .ktext base address               |
| 0x00003fff | user space high address           |
| 0x00003ffc | text limit address                |
| 0x00003000 | .text base address                |
| 0x00002fff | data segment limit address        |
| 0x00002ffc | stack pointer \$sp                |
| 0x00002ffc | stack base address                |
| 0x00002000 | stack limit address               |
| 0x00002000 | heap base address                 |
| 0x00001800 | global pointer \$gp               |
| 0x00001000 | .extern base address              |
| 0x00000000 | data segment base address         |
| 0x00000000 | .data base address                |

Apply and Close

Apply

Cancel

Reset

第二步：在 MARS 汇编仿真器中打开 sum\_mips.asm

EditExecute

helloworld\_mips.asm

sum\_mips.asm

```

1  #MIPS程序 求累加和                                sum_mips.asm
2  #求累加和: 1+2+.....+n, n的值为10 (可以改变), 累加和的结果存放到地址为0的数据存储器中
3
4  main:
5      addi $s0,$zero,10                                # n=10 -> s0
6      addi $s1,$zero,1                                #      1 -> s1
7      addi $s2,$zero,1                                #      1 -> s2
8      addi $s3,$zero,0                                #      0 -> s3
9  loop:
10     add $s3,$s3,$s1                                # s3+s1 -> s3
11     beq $s1,$s0,finish                             # 如果s1=s0, 则转finish
12     add $s1,$s1,$s2                                # s1+s2 -> s1
13     j loop
14  finish:
15     sw $s3,0($zero)                                # s3 存到地址为0的存储单元中
16
17     addi $v0,$zero,10                                # 10号系统调用
18     syscall                                          # 程序退出
19
20

```

第三步：对 sum\_mips.asm 源程序进行汇编

| Edit Execute             |            |            |                          |                                              |
|--------------------------|------------|------------|--------------------------|----------------------------------------------|
| Text Segment             |            |            |                          |                                              |
| Bkpt                     | Address    | Code       | Basic                    | Source                                       |
| <input type="checkbox"/> | 0x00003000 | 0x2010000a | addi \$16,\$0,0x00000... | 5: addi \$s0,\$zero,10 # n=10 -> s0          |
| <input type="checkbox"/> | 0x00003004 | 0x20110001 | addi \$17,\$0,0x00000... | 6: addi \$s1,\$zero,1 # 1 -> s1              |
| <input type="checkbox"/> | 0x00003008 | 0x20120001 | addi \$18,\$0,0x00000... | 7: addi \$s2,\$zero,1 # 1 -> s2              |
| <input type="checkbox"/> | 0x0000300c | 0x20130000 | addi \$19,\$0,0x00000... | 8: addi \$s3,\$zero,0 # 0 -> s3              |
| <input type="checkbox"/> | 0x00003010 | 0x02719820 | add \$19,\$19,\$17       | 10: add \$s3,\$s3,\$s1 # s3+s1 -> s3         |
| <input type="checkbox"/> | 0x00003014 | 0x12300002 | beq \$17,\$16,0x00000... | 11: beq \$s1,\$s0,finish # 如果s1=s0, 则转finish |
| <input type="checkbox"/> | 0x00003018 | 0x02328820 | add \$17,\$17,\$18       | 12: add \$s1,\$s1,\$s2 # s1+s2 -> s1         |
| <input type="checkbox"/> | 0x0000301c | 0x08000c04 | j 0x00003010             | 13: j loop                                   |
| <input type="checkbox"/> | 0x00003020 | 0xac130000 | sw \$19,0x00000000(\$0)  | 15: sw \$s3,0(\$zero) # s3 存到地址为0的存储单元中      |
| <input type="checkbox"/> | 0x00003024 | 0x2002000a | addi \$2,\$0,0x0000000a  | 17: addi \$v0,\$zero,10 # 10号系统调用            |
| <input type="checkbox"/> | 0x00003028 | 0x0000000c | syscall                  | 18: syscall # 程序退出                           |

第四步：运行汇编后的程序

```

Mars Messages Run I/O
Hello, World!

-- program is finished running --

Reset: reset completed.

Clear
Hello, World!

-- program is finished running --

-- program is finished running --

```

```

Mars Messages Run I/O
Step: execution completed successfully.

Assemble: assembling E:\lab4\mips\helloworld_mips.asm

Assemble: operation completed successfully.

Assemble: assembling E:\lab4\mips\sum_mips.asm

Clear
Assemble: operation completed successfully.

Go: running sum_mips.asm

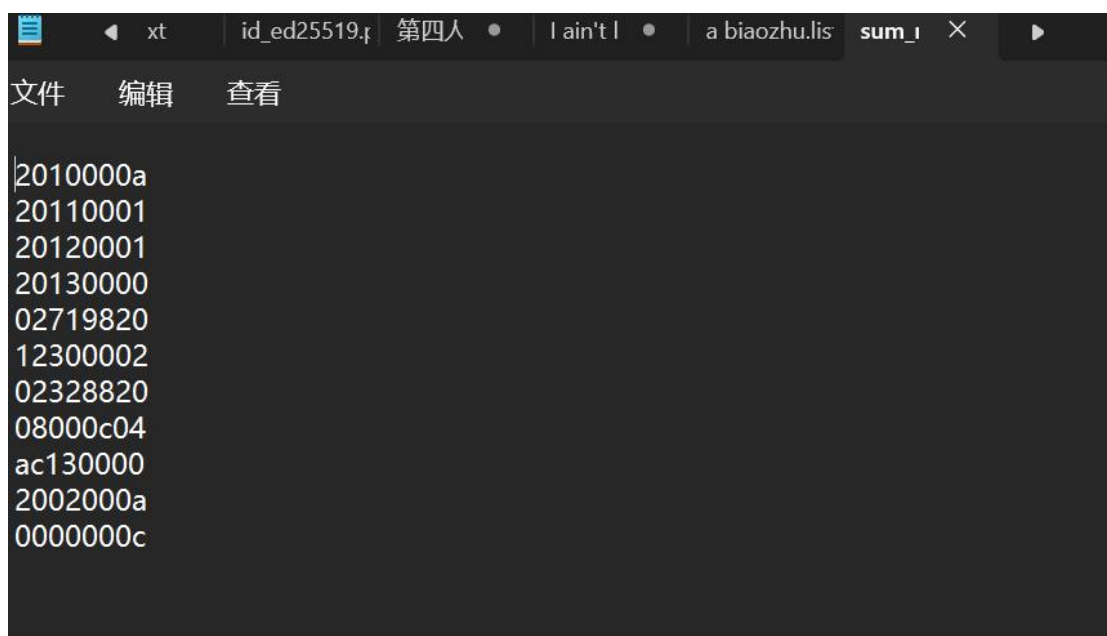
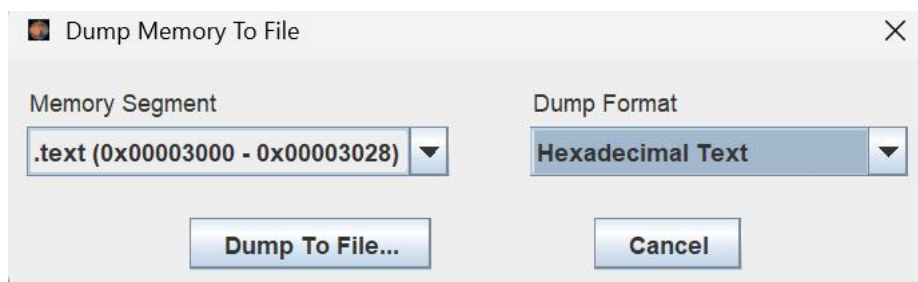
Go: execution completed successfully.

```

第五步：观看数据段中的内容（地址=0）是否为 37h=1+2+...+10=55？

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00000000   | 0x00000037 |
| 0x00000020   | 0x00000000 |
| 0x00000040   | 0x00000000 |

第六步：导出汇编后的机器码





| Edit Execute             |            |            |    |
|--------------------------|------------|------------|----|
| Text Segment             |            |            |    |
| Bkpt                     | Address    | Code       |    |
| <input type="checkbox"/> | 0x00003000 | 0x2010000a | ad |
| <input type="checkbox"/> | 0x00003004 | 0x20110001 | ad |
| <input type="checkbox"/> | 0x00003008 | 0x20120001 | ad |
| <input type="checkbox"/> | 0x0000300c | 0x20130000 | ad |
| <input type="checkbox"/> | 0x00003010 | 0x02719820 | ad |
| <input type="checkbox"/> | 0x00003014 | 0x12300002 | be |
| <input type="checkbox"/> | 0x00003018 | 0x02328820 | ad |
| <input type="checkbox"/> | 0x0000301c | 0x08000c04 | j  |
| <input type="checkbox"/> | 0x00003020 | 0xac130000 | sw |
| <input type="checkbox"/> | 0x00003024 | 0x2002000a | ad |
| <input type="checkbox"/> | 0x00003028 | 0x0000000c | sy |

d. 计算斐波那契数列的 MIPS 汇编语言程序

Mars Messages Run I/O

Go: running sum\_mips.asm

Go: execution completed successfully.

Assemble: assembling E:\lab4\mips\fib\_mips.asm

Assemble: operation completed successfully.

Go: running fib\_mips.asm

Go: execution completed successfully.

Clear

```

2012000a
20110000
ac110000
20110001
ac110004
ac110008
20130002
20140001
20150001
0295b020
22b40000
22d50000
22730001
0273b820
02f78020
ae160000
12530001
08000c09
2002000a
0000000c

```

| Edit                                                                                                    |            | Execute    |   |
|---------------------------------------------------------------------------------------------------------|------------|------------|---|
|  <b>Text Segment</b> |            |            |   |
| Bkpt                                                                                                    | Address    | Code       |   |
|                      | 0x00003000 | 0x2012000a | a |
|                      | 0x00003004 | 0x20110000 | a |
|                      | 0x00003008 | 0xac110000 | s |
|                      | 0x0000300c | 0x20110001 | a |
|                      | 0x00003010 | 0xac110004 | s |
|                      | 0x00003014 | 0xac110008 | s |
|                      | 0x00003018 | 0x20130002 | a |
|                      | 0x0000301c | 0x20140001 | a |
|                      | 0x00003020 | 0x20150001 | a |
|                      | 0x00003024 | 0x0295b020 | a |
|                      | 0x00003028 | 0x22b40000 | a |
|                      | 0x0000302c | 0x22d50000 | a |
|                      | 0x00003030 | 0x22730001 | a |
|                      | 0x00003034 | 0x0273b820 | a |
|                      | 0x00003038 | 0x02f78020 | a |
|                      | 0x0000303c | 0xae160000 | s |
|                      | 0x00003040 | 0x12530001 | b |
|                      | 0x00003044 | 0x08000c09 | j |
|                      | 0x00003048 | 0x2002000a | a |
|                      | 0x0000304c | 0x0000000c | s |

- e. 修改“sum\_mips.asm”程序，计算  $1+2+3+\dots+100=5050=13bah$ （修改后的程序命名为： sum100\_mips.asm），然后在 MARS 4.5 汇编仿真器中运行，观看运算结果是否正

确？并将该程序的机器码保存到 sum100\_mips.hex 文件中。

| Edit                | Execute                                            |
|---------------------|----------------------------------------------------|
| helloworld_mips.asm | sum_mips.asm   fib_mips.asm <b>sum100_mips.asm</b> |

```

1  #MIPS程序 求累加和                                     sum_mips.asm
2  #求累加和: 1+2+.....+n, n的值为10 (可以改变), 累加和的结果存放到地址为0的数据存储器中
3
4  main:
5      addi $s0,$zero,100                                     # n=100 -> s0
6      addi $s1,$zero,1                                       #      1 -> s1
7      addi $s2,$zero,1                                       #      1 -> s2
8      addi $s3,$zero,0                                       #      0 -> s3
9
10     loop:
11         add $s3,$s3,$s1                                     # s3+s1 -> s3
12         beq $s1,$s0,finish                                 # 如果s1=s0, 则转finish
13         add $s1,$s1,$s2                                     # s1+s2 -> s1
14         j loop
15     finish:
16         sw $s3,0($zero)                                     # s3 存到地址为0的存储单元中
17
18         addi $v0,$zero,10                                   # 10号系统调用
19         syscall                                             # 程序退出
20

```

| Mars Messages    | Run I/O                                           |
|------------------|---------------------------------------------------|
| <div>Clear</div> | Go: running fib_mips.asm                          |
|                  | Go: execution completed successfully.             |
|                  | Assemble: assembling E:\lab4\mips\sum100_mips.asm |
|                  | Assemble: operation completed successfully.       |
|                  | Go: running sum100_mips.asm                       |
|                  | Go: execution completed successfully.             |

| Data Segment |            |            |
|--------------|------------|------------|
| Address      | Value (+0) | Value      |
| 0x00000000   | 0x000013ba | 0x000013ba |
| 0x00000020   | 0x00000000 | 0x00000000 |



The screenshot shows a web browser window with several tabs. The active tab is titled 'sum1l'. The browser's address bar shows 'a biao zhu . lis | sum\_mips . he | fib\_mips . hex'. The page has a menu bar with '文件' (File), '编辑' (Edit), and '查看' (View). The main content area displays the following assembly code:

```

20100064
20110001
20120001
20130000
02719820
12300002
02328820
08000c04
ac130000
2002000a
0000000c

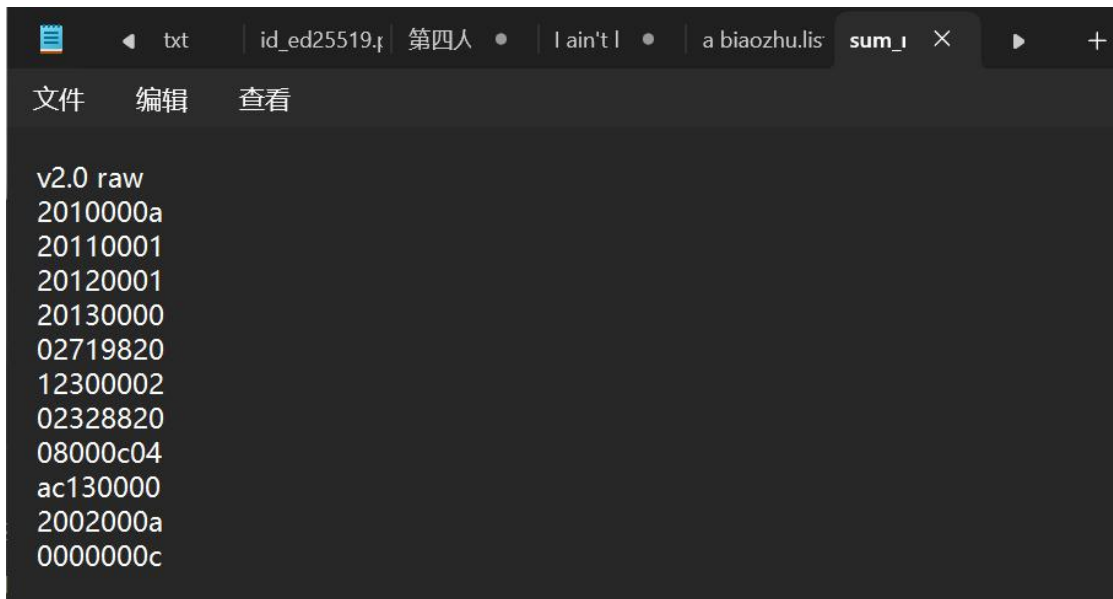
```

| Text Segment             |            |            |        |
|--------------------------|------------|------------|--------|
| Bkpt                     | Address    | Code       |        |
| <input type="checkbox"/> | 0x00003000 | 0x20100064 | addi   |
| <input type="checkbox"/> | 0x00003004 | 0x20110001 | addi   |
| <input type="checkbox"/> | 0x00003008 | 0x20120001 | addi   |
| <input type="checkbox"/> | 0x0000300c | 0x20130000 | addi   |
| <input type="checkbox"/> | 0x00003010 | 0x02719820 | add s  |
| <input type="checkbox"/> | 0x00003014 | 0x12300002 | beq s  |
| <input type="checkbox"/> | 0x00003018 | 0x02328820 | add s  |
| <input type="checkbox"/> | 0x0000301c | 0x08000c04 | j 0x0  |
| <input type="checkbox"/> | 0x00003020 | 0xac130000 | sw \$s |
| <input type="checkbox"/> | 0x00003024 | 0x2002000a | addi   |
| <input type="checkbox"/> | 0x00003028 | 0x0000000c | sysca  |

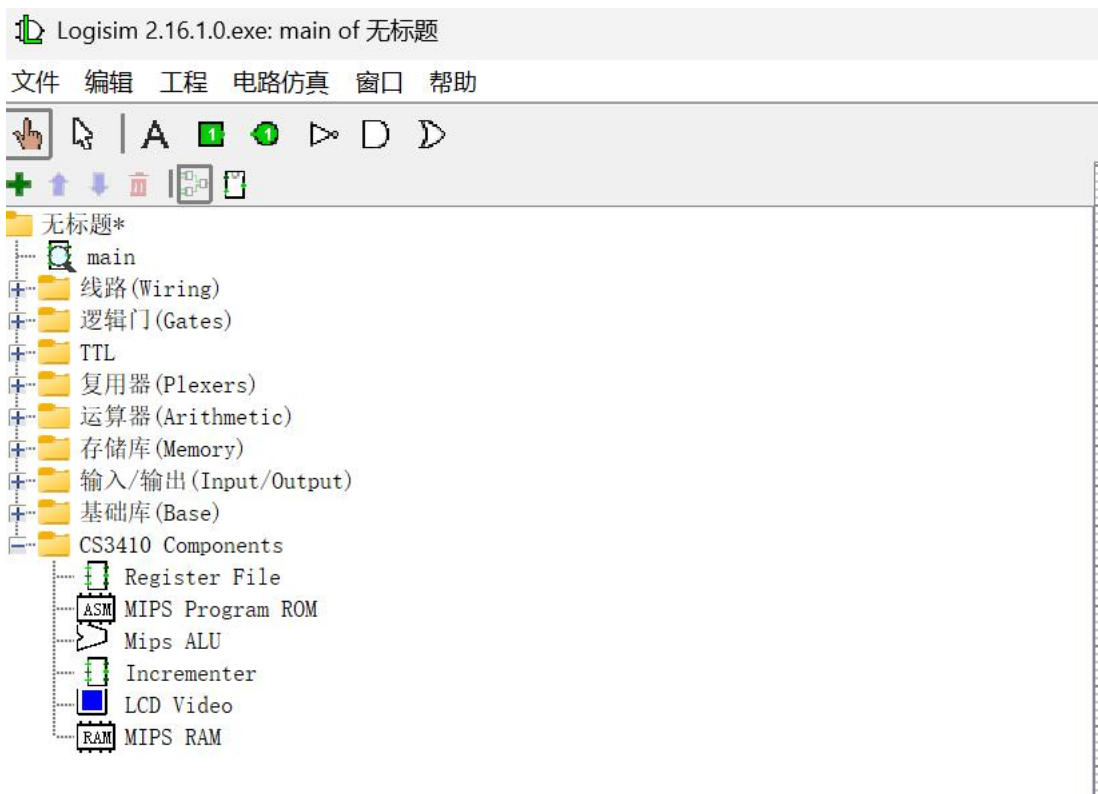
## (2) 在 Logisim 上运行 MIPS 程序

### a. 在 Logisim 实现的单周期 MIPS 处理器上运行求累加和程序

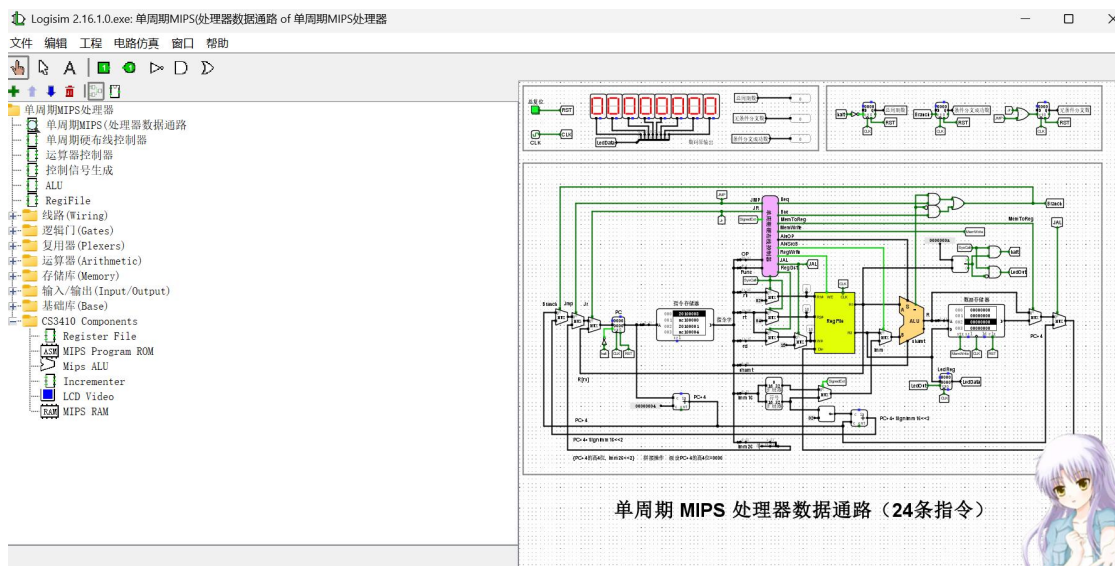
第一步：修改前面导出的机器码文件 sum\_mips.hex，增加： v2.0 raw



第二步：运行 Logisim，在 Logisim 中增加寄存器文件库 “cs3410.jar”



第三步：在 Logisim 中打开设计文件 “单周期 MIPS 处理器.circ”



第四步: 在单周期 MIPS 处理器数据通路的指令存储器中装入机器码文件 sum\_mips.hex

```

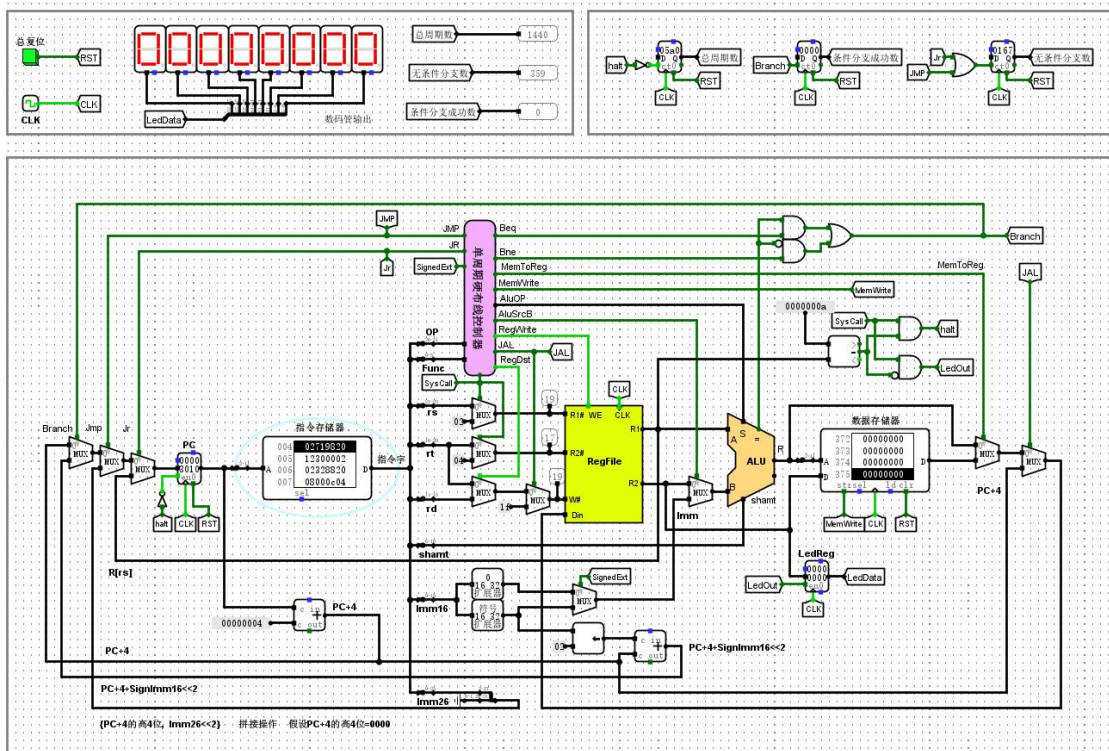
000 2010000a 20110001 20120001 20130000 02719820 12300002 02328820 08000c04 ac130000 2002000a 0000000c 00000000 00000000 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

第五步: 设置时钟频率=1KHz, 按 Ctrl+K 启动时钟, 程序开始执行

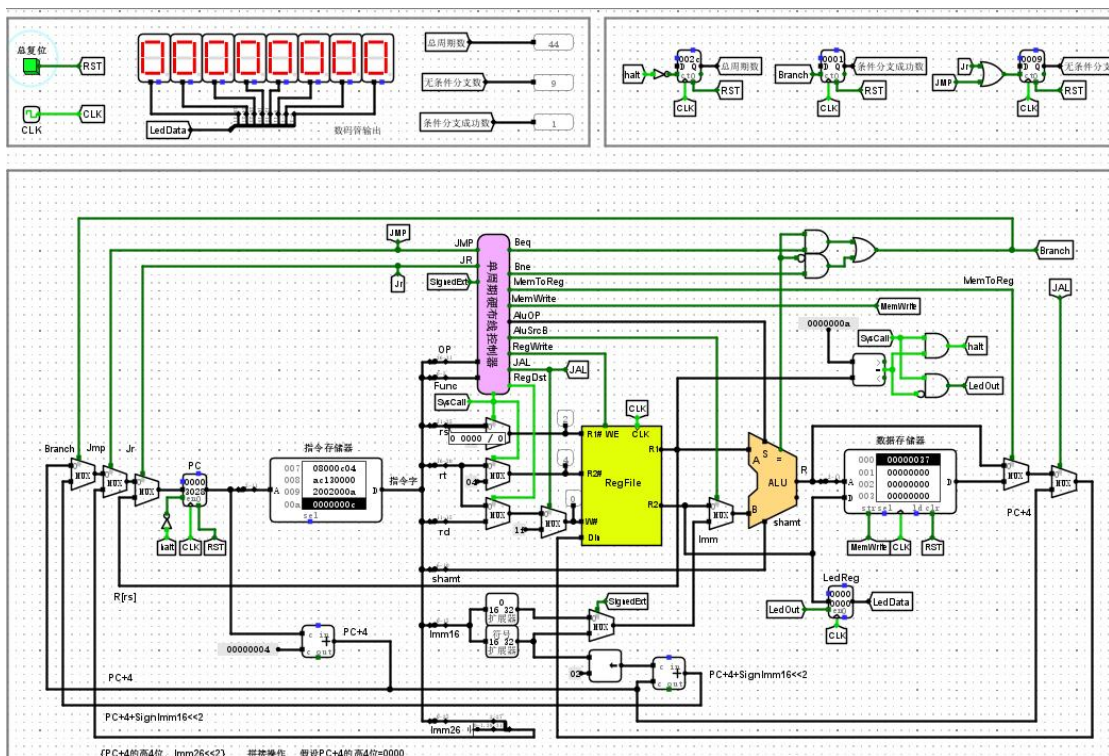




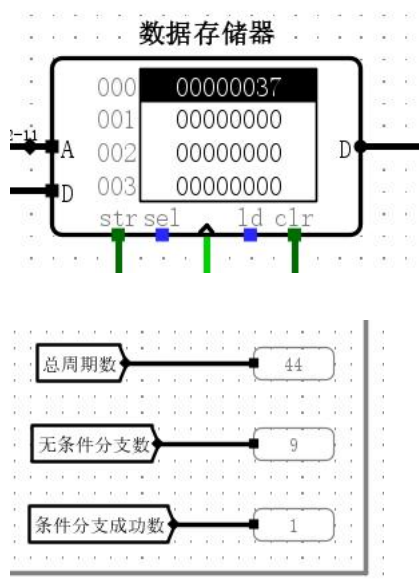


## 单周期 MIPS 处理器数据通路（24条指令）

第六步：按总复位，程序重新执行；指令存储器中的内容停止滚动后，表示程序执行结束；再按 Ctrl+K 停止时钟



第七步：此时观察数据存储器第 0 号单元的值（累加和，正确值为 37h=55），以及程序运行的总周期数、无条件分支数、条件分支成功数



b. 在 Logisim 实现的单周期 MIPS 处理器上运行计算费波那契数列程序

第一步：修改前面导出的机器码文件 fib\_mips.hex，增加： v2.0 raw

```

v2.0 raw
2012000a
20110000
ac110000
20110001
ac110004
ac110008
20130002
20140001
20150001
0295b020
22b40000
22d50000
22730001
0273b820
02f78020
ae160000
12530001
08000c09
2002000a
0000000c
  
```

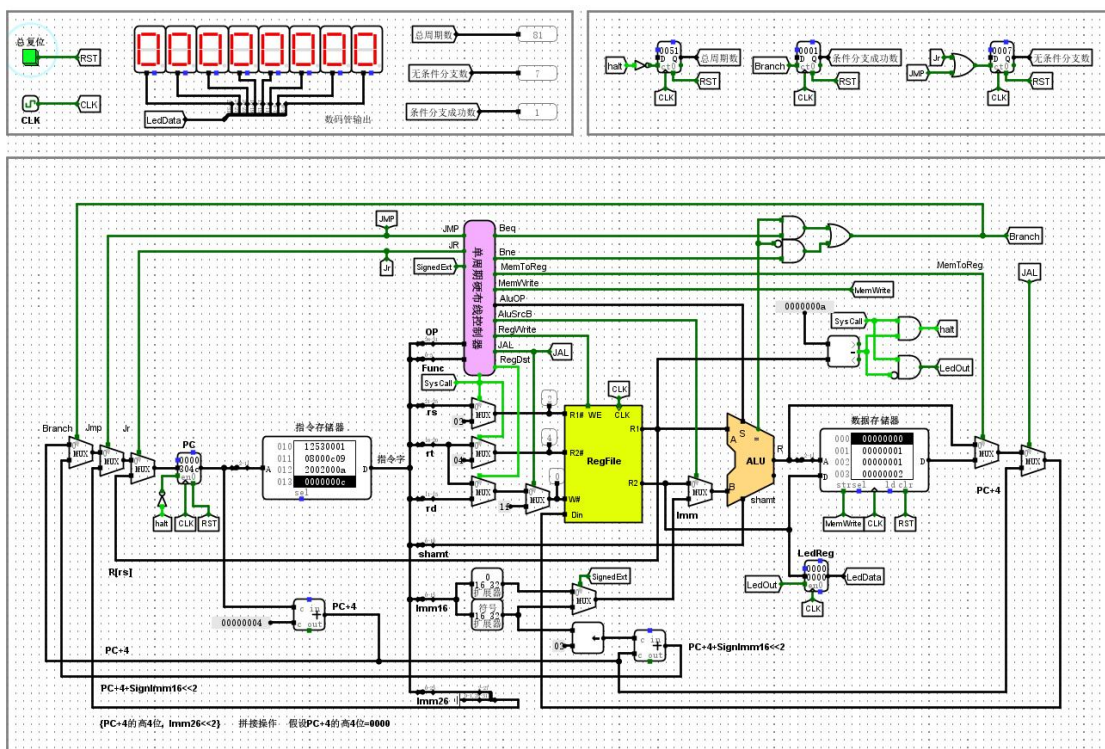


第二步：在单周期 MIPS 处理器数据通路的指令存储器中装入机器码文件 fib.mips.hex

[illegible]

第三步：设置时钟频率=1KHz，按 Ctrl+K 启动时钟，程序开始执行

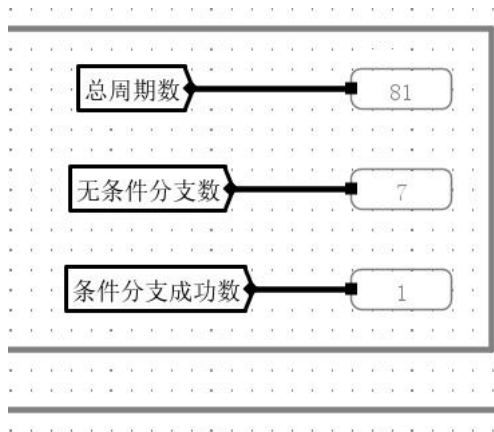
第四步：按总复位，程序重新执行；指令存储器中的内容停止滚动后，表示程序执行结束；再按 Ctrl+K 停止时钟



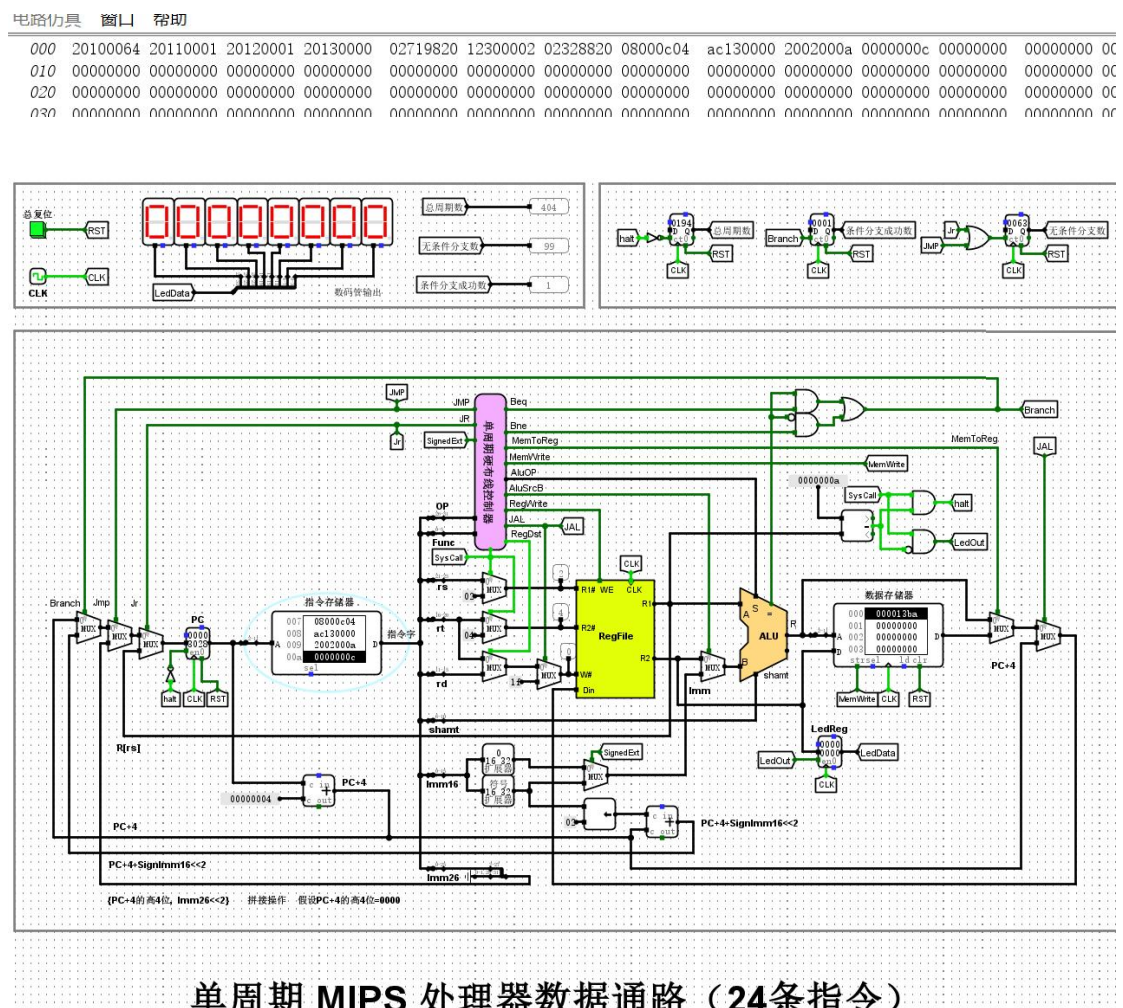
## 单周期 MIPS 处理器数据通路（24条指令）

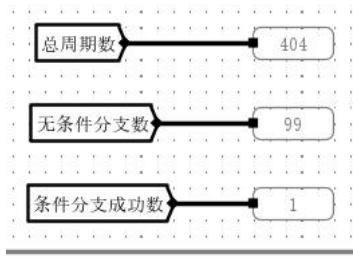
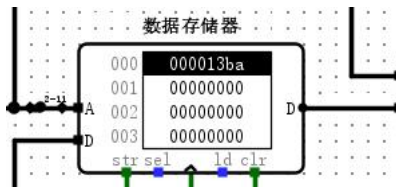
第五步：此时观察数据存储器中的内容（0 号单元开始的 11 个单元），以及程序运行的总周期数、无条件分支数、条件分支成功数

[illegible]



c. 在 Logisim 实现的单周期 MIPS 处理器上，运行计算  $1+2+3+\dots+100=5050=13\text{bah}$  的机器语言程序 (sum100\_mips.hex)，观看数据存储器 0 号单元的值是不是 13bah？程序运行的总周期数、无条件分支数、条件分支成功数分别是多少？

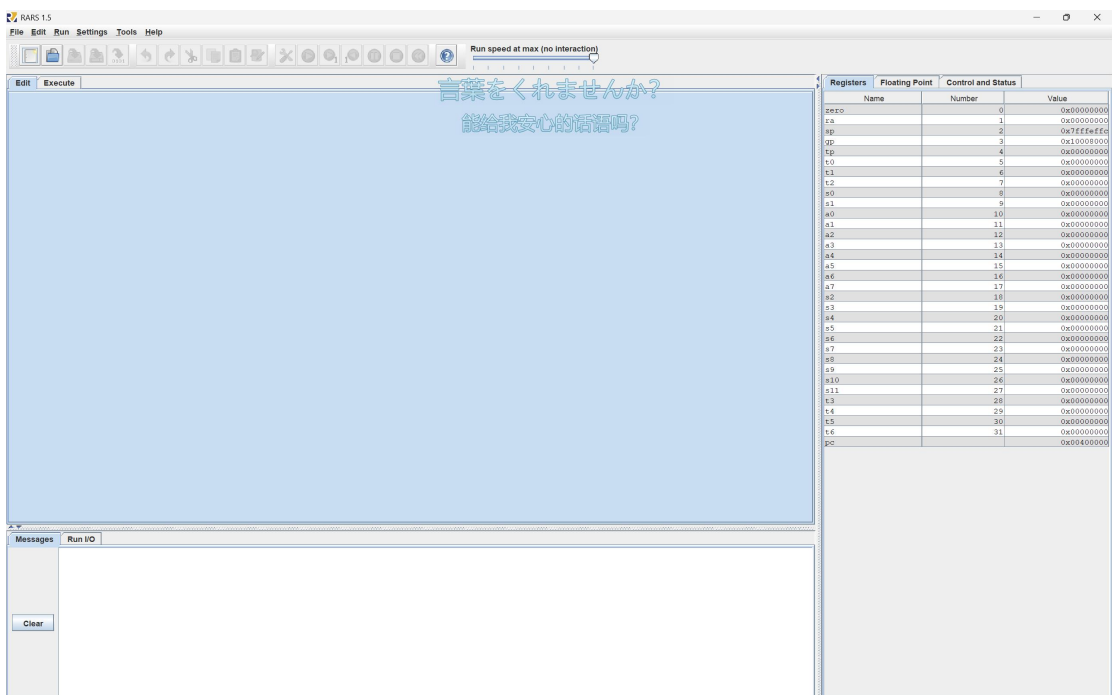




### (3) RISC-V 汇编语言程序的运行

#### a. 运行 RISC-V 汇编仿真器

运行 RISC-V 汇编仿真器 rars1\_5.jar



#### b. 运行 RISC-V 求累加和程序

利用前面的 RISC-V 的 9 条指令，编写一个计算  $1+2+\dots+n$  的累加和程序，

结果保存到数据存储器地址为 0 的存储单元中；该程序的汇编代码如下（sum\_riscv.asm，位于\ riscv 目录中）：



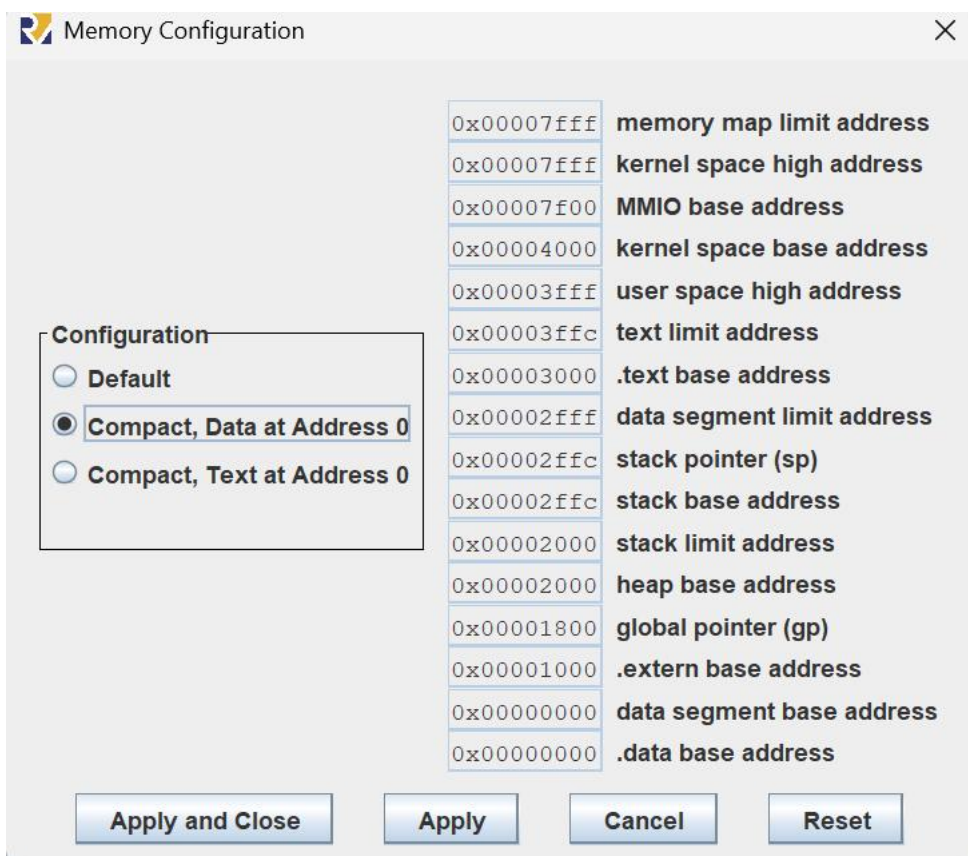
```

# RISC-V riscv.asm      sum riscv.asm      result=1+2+...+n
# result 0x00000000 0x00000000 0x00000000

main:
    ori a3, zero, 10      # a3=10=n
    ori a5, zero, 1        # a5=1
    ori a2, zero, 1        # a2=1
    ori a4, zero, 0        # a4=0
loop:
    add a4, a4, a5        # a4+a5 -> a4
    beq a5, a3, finish    # a5=a3 0x00000000 0x00000000
    add a5, a5, a2        # a5+a2 -> a5
    jal zero, loop        # 0x00000000 0x00000000
finish:
    sw a4, 0(zero)        # a4 -> (0) 0x00000000 0x00000000
end:
    jal zero, end        # 0x00000000 0x00000000

```

第一步：将 RARS 汇编仿真器的数据段的开始地址设置为 0



第二步：在 RARS 中打开“sum\_riscv.asm”

```

Edit Execute
sum_riscv.asm
1 # RISC-V sum_riscv.asm result=1+2+...+n
2 # result
3
4 main:
5     ori a3,zero,10      # a3=10=n
6     ori a5, zero, 1     # a5=1
7     ori a2, zero, 1     # a2=1
8     ori a4, zero, 0     # a4=0
9 loop:
10    add a4, a4, a5       # a4+a5 -> a4
11    beq a5, a3, finish   # a5=a3
12    add a5, a5, a2       # a5+a2 -> a5
13    jal zero, loop       # loop
14 finish:
15    sw a4, 0(zero)       # a4 -> (0)
16 end:
17    jal zero, end        # end
18

```

第三步：在 RARS 中汇编“sum\_riscv.asm”

| Text Segment             |            |            |                        |        |                              |
|--------------------------|------------|------------|------------------------|--------|------------------------------|
| Bkpt                     | Address    | Code       | Basic                  | Source |                              |
| <input type="checkbox"/> | 0x00003000 | 0x00a06693 | ori x13,x0,0x0000000a  | 5:     | ori a3,zero,10 # a3=10=n     |
| <input type="checkbox"/> | 0x00003004 | 0x00106793 | ori x15,x0,0x00000001  | 6:     | ori a5, zero, 1 # a5=1       |
| <input type="checkbox"/> | 0x00003008 | 0x00106613 | ori x12,x0,0x00000001  | 7:     | ori a2, zero, 1 # a2=1       |
| <input type="checkbox"/> | 0x0000300c | 0x00006713 | ori x14,x0,0x00000000  | 8:     | ori a4, zero, 0 # a4=0       |
| <input type="checkbox"/> | 0x00003010 | 0x00f70733 | add x14,x14,x15        | 10:    | add a4, a4, a5 # a4+a5 -> a4 |
| <input type="checkbox"/> | 0x00003014 | 0x00d78663 | beq x15,x13,0x00000000 | 11:    | beq a5, a3, finish # a5=a3   |
| <input type="checkbox"/> | 0x00003018 | 0x00c787b3 | add x15,x15,x12        | 12:    | add a5, a5, a2 # a5+a2 -> a5 |
| <input type="checkbox"/> | 0x0000301c | 0xf5f06df  | jal x0,0xffffffff      | 13:    | jal zero, loop # loop        |
| <input type="checkbox"/> | 0x00003020 | 0x00e02023 | sw x14,0x00000000(x0)  | 15:    | sw a4, 0(zero) # a4 -> (0)   |
| <input type="checkbox"/> | 0x00003024 | 0x0000006f | jal x0,0x00000000      | 17:    | jal zero, end # end          |

Messages
Run I/O

Assemble: assembling E:\lab4\riscv\sum\_riscv.asm  
Assemble: operation completed successfully.

Clear

第四步：在 RARS 中运行“sum\_riscv.asm”

在最后一指令上设置断点

| Text Segment                        |            |            |                        |     |
|-------------------------------------|------------|------------|------------------------|-----|
| Bkpt                                | Address    | Code       | Basic                  |     |
| <input type="checkbox"/>            | 0x00003000 | 0x00a06693 | ori x13,x0,0x0000000a  | 5:  |
| <input type="checkbox"/>            | 0x00003004 | 0x00106793 | ori x15,x0,0x00000001  | 6:  |
| <input type="checkbox"/>            | 0x00003008 | 0x00106613 | ori x12,x0,0x00000001  | 7:  |
| <input type="checkbox"/>            | 0x0000300c | 0x00006713 | ori x14,x0,0x00000000  | 8:  |
| <input type="checkbox"/>            | 0x00003010 | 0x00f70733 | add x14,x14,x15        | 10: |
| <input type="checkbox"/>            | 0x00003014 | 0x00d78663 | beq x15,x13,0x000000.. | 11: |
| <input type="checkbox"/>            | 0x00003018 | 0x00c787b3 | add x15,x15,x12        | 12: |
| <input type="checkbox"/>            | 0x0000301c | 0xff5ff06f | jal x0,0xfffffffffa    | 13: |
| <input type="checkbox"/>            | 0x00003020 | 0x00e02023 | sw x14,0x00000000(x0)  | 15: |
| <input checked="" type="checkbox"/> | 0x00003024 | 0x0000006f | jal x0,0x00000000      | 17: |

运行

| Text Segment                        |            |            |                        |     | Source             |               |
|-------------------------------------|------------|------------|------------------------|-----|--------------------|---------------|
| Bkpt                                | Address    | Code       | Basic                  |     |                    |               |
| <input type="checkbox"/>            | 0x00003000 | 0x00a06693 | ori x13,x0,0x0000000a  | 5:  | ori a3,zero,10     | # a3=10=n     |
| <input type="checkbox"/>            | 0x00003004 | 0x00106793 | ori x15,x0,0x00000001  | 6:  | ori a5, zero, 1    | # a5=1        |
| <input type="checkbox"/>            | 0x00003008 | 0x00106613 | ori x12,x0,0x00000001  | 7:  | ori a2, zero, 1    | # a2=1        |
| <input type="checkbox"/>            | 0x0000300c | 0x00006713 | ori x14,x0,0x00000000  | 8:  | ori a4, zero, 0    | # a4=0        |
| <input type="checkbox"/>            | 0x00003010 | 0x00f70733 | add x14,x14,x15        | 10: | add a4, a4, a5     | # a4+a5 -> a4 |
| <input type="checkbox"/>            | 0x00003014 | 0x00d78663 | beq x15,x13,0x000000.. | 11: | beq a5, a3, finish | # a5=a3       |
| <input type="checkbox"/>            | 0x00003018 | 0x00c787b3 | add x15,x15,x12        | 12: | add a5, a5, a2     | # a5+a2 -> a5 |
| <input type="checkbox"/>            | 0x0000301c | 0xff5ff06f | jal x0,0xfffffffffa    | 13: | jal zero, loop     | # loop        |
| <input type="checkbox"/>            | 0x00003020 | 0x00e02023 | sw x14,0x00000000(x0)  | 15: | sw a4, 0(zero)     | # a4 -> (0)   |
| <input checked="" type="checkbox"/> | 0x00003024 | 0x0000006f | jal x0,0x00000000      | 17: | jal zero, end      | # end         |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00000000   | 0x00000037 |
| 0x00000020   | 0x00000000 |
| 0x00000040   | 0x00000000 |
| 0x00000060   | 0x00000000 |

第五步：将汇编后的机器码，导出到一个文本文件中

Dump Memory To File

Memory Segment

.text (0x00003000 - 0x00003024)

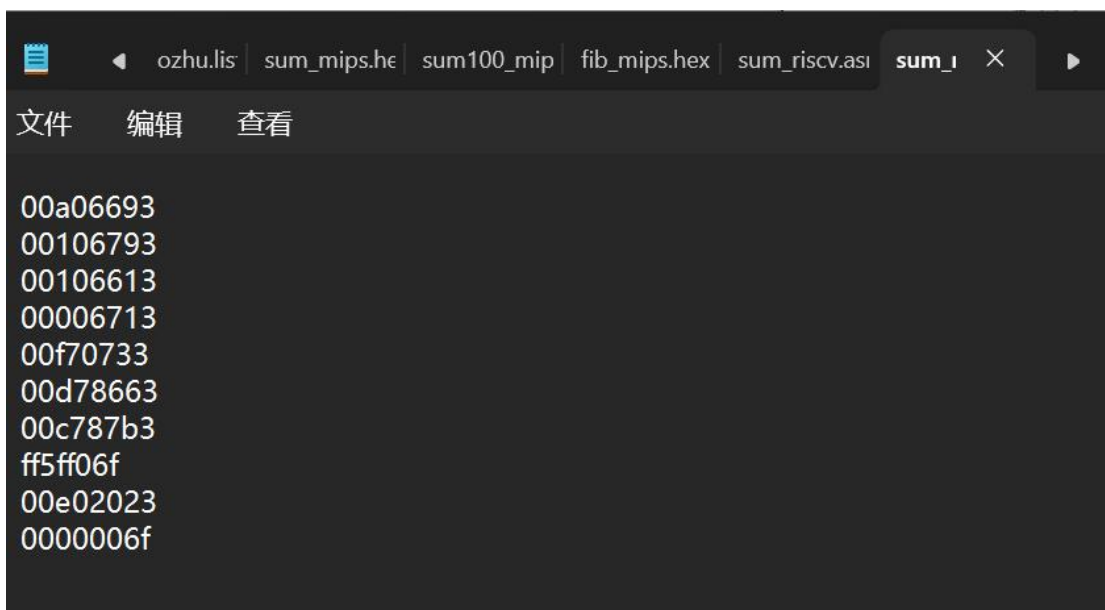
Dump Format

Hexadecimal Text

Dump To File...

Cancel





c. 运行 RISC-V 计算费波那契数列的程序

```

Edit Execute
sum_riscv.asm fib_riscv.asm
1 #RISC-V程序 计算费波那契数列 fib_riscv.asm
2 # n为数据个数 (可以改变)
3 # 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 存放在地址为0、4、8、12、...4*n的数据存储器中
4
5 main:
6     ori a2, zero, 10          # a2 = 10 = n
7
8     ori a1, zero, 0           # a1 = 0
9     sw a1, 0(zero)            # 0 -> (0)
10
11    ori a1, zero, 1           # a1 = 1
12    sw a1, 4(zero)            # 1 -> (4)
13
14    sw a1, 8(zero)            # 1 -> (8)
15
16    ori a3, zero, 2           # a3 = 2
17    ori a4, zero, 1           # a4 = 1
18    ori a5, zero, 1           # a5 = 1
19
20 loop:
21    add a6, a4, a5            # a4+a5 -> a6
22    ori a4, a5, 0             # a5 -> a4
23    ori a5, a6, 0             # a6 -> a5
24
25    add a3, a3, a1            # a3+1 -> a3      a1=1
26    add a7, a3, a3            # a3*2 -> a7
27    add a0, a7, a7            # a7*2 -> a0
28    sw a6, 0(a0)              # a6 -> 0(a0)
29    beq a2, a3, end           # if a2=a3 goto end
30    jal zero, loop            # goto loop
31
32 end:
33    jal zero, end             # goto end

```

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x00000000 | 0x00000000 | 0x00000001 | 0x00000001 | 0x00000002 | 0x00000003  | 0x00000005  | 0x00000008  | 0x0000000d  |
| 0x00000020 | 0x00000015 | 0x00000022 | 0x00000037 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000040 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000060 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

```

mips.h sum100_mip fib_mips.hex sum_riscv.asi sum_riscv.he fib_ris
文件 编辑 查看

00a06613
00006593
00b02023
00106593
00b02223
00b02423
00206693
00106713
00106793
00f70833
0007e713
00086793
00b686b3
00d688b3
01188533
01052023
00d60463
fe1ff06f
0000006f

```

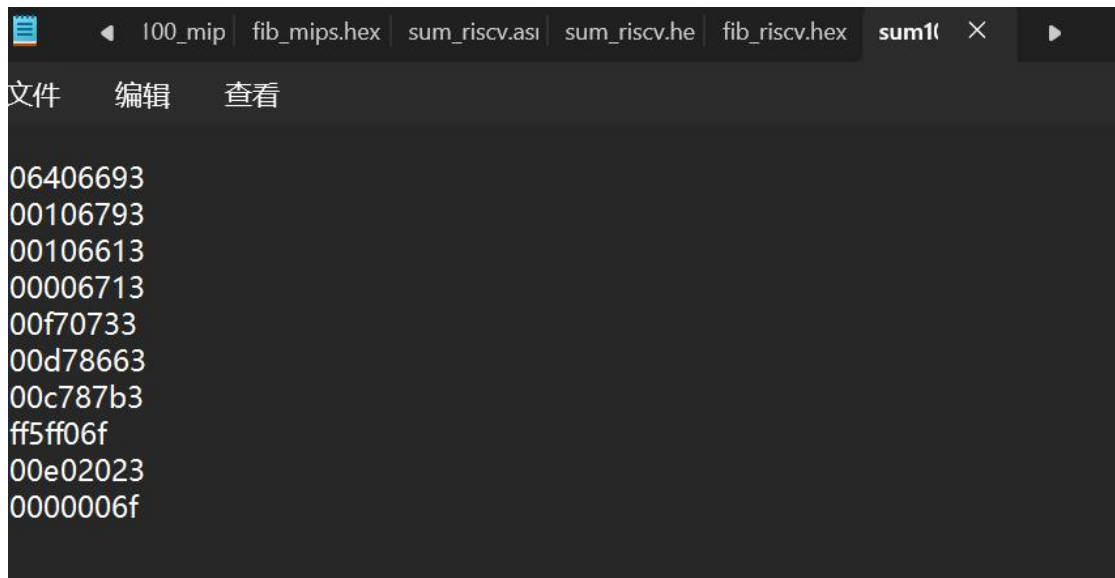
d. 修改 sum\_riscv.asm 程序, 计算  $1+2+3+\dots+100=5050=13bah$  (修改后的程序命名为: sum100\_riscv.asm), 然后在 RARS 1.5 汇编仿真器中运行, 并将该程序的机器码保存到 sum100\_riscv.hex 文件中。

```

Edit Execute
sum100_riscv.asm fib_riscv.asm*
1 # RISC-V sum_riscv.asm result=1+2+...+n
2 # result
3
4 main:
5     ori a3,zero,100      # a3=100=n
6     ori a5, zero, 1      # a5=1
7     ori a2, zero, 1      # a2=1
8     ori a4, zero, 0      # a4=0
9 loop:
10    add a4, a4, a5        # a4+a5 -> a4
11    beq a5, a3, finish    # a5=a3
12    add a5, a5, a2        # a5+a2 -> a5
13    jal zero, loop        # loop
14 finish:
15    sw a4, 0(zero)        # a4 -> (0)
16 end:
17    jal zero, end         # end
18

```

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00000000   | 0x000013ba |
| 0x00000020   | 0x00000000 |
| 0x00000040   | 0x00000000 |
| 0x00000060   | 0x00000000 |



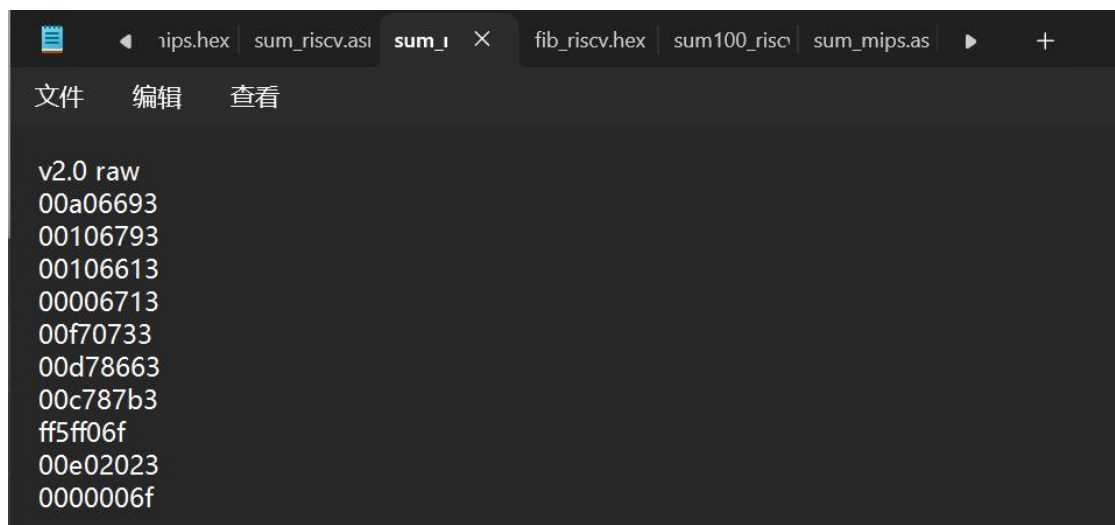
A screenshot of a text editor window with a dark theme. The title bar shows several open files: 100\_mip, fib\_mips.hex, sum\_riscv.asi, sum\_riscv.he, fib\_riscv.hex, and sum1l. The editor has a menu bar with '文件' (File), '编辑' (Edit), and '查看' (View). The main text area contains a list of hexadecimal values, each on a new line: 06406693, 00106793, 00106613, 00006713, 00f70733, 00d78663, 00c787b3, ff5ff06f, 00e02023, and 0000006f.

```
06406693
00106793
00106613
00006713
00f70733
00d78663
00c787b3
ff5ff06f
00e02023
0000006f
```

#### (4) 在 Logisim 上运行 RISC-V 程序

##### a. 在 Logisim 实现的单周期 RISC-V 处理器上运行求累加和的程序

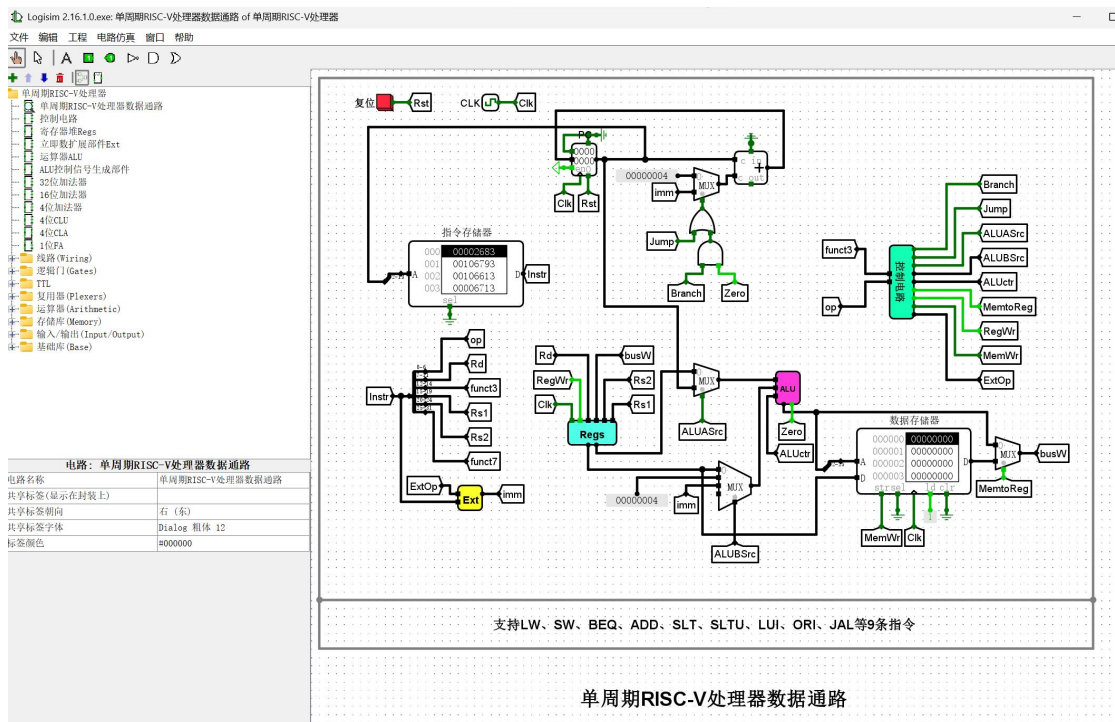
第一步：修改前面导出的机器码文件 `sum_riscv.hex`，增加： `v2.0 raw`



A screenshot of a text editor window with a dark theme. The title bar shows several open files: mips.hex, sum\_riscv.asi, sum\_r, fib\_riscv.hex, sum100\_risc, and sum\_mips.as. The editor has a menu bar with '文件' (File), '编辑' (Edit), and '查看' (View). The main text area contains the same list of hexadecimal values as the previous screenshot, but with 'v2.0 raw' added at the very beginning of the list.

```
v2.0 raw
00a06693
00106793
00106613
00006713
00f70733
00d78663
00c787b3
ff5ff06f
00e02023
0000006f
```

第二步：在 Logisim 中打开设计文件“单周期 RISC-V 处理器.circ”



第三步：按 Ctrl+R

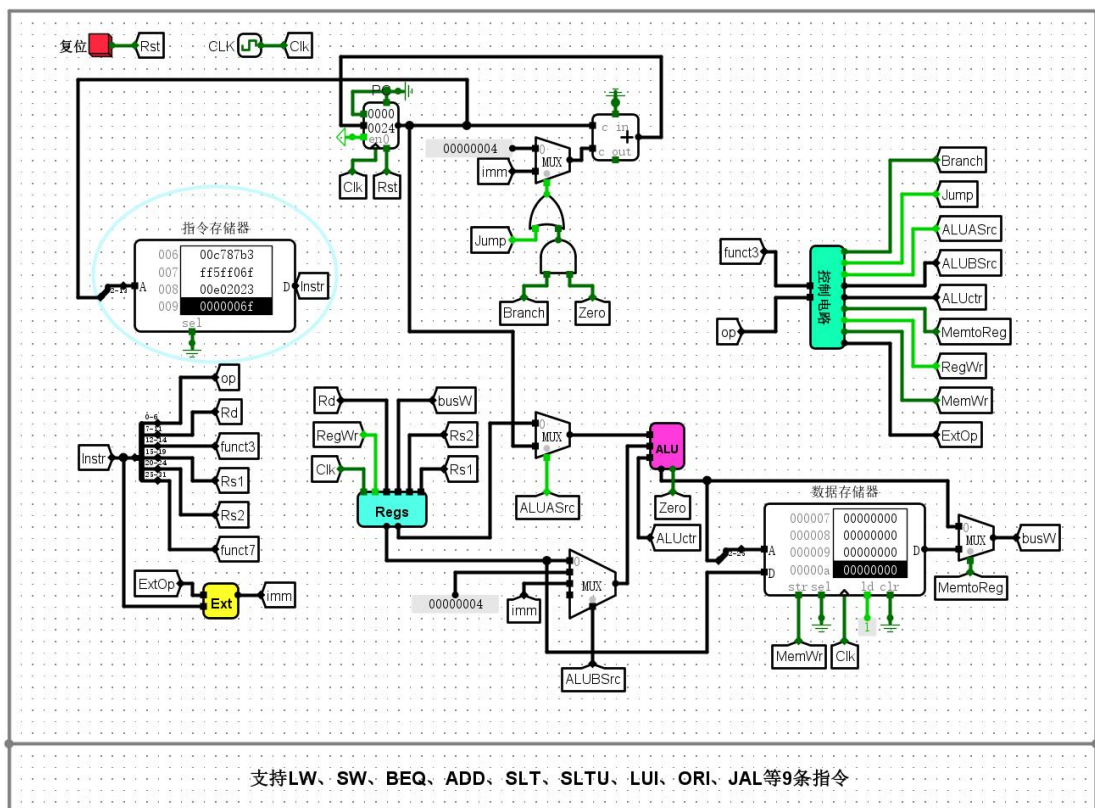
第四步：在单周期 RISC-V 处理器数据通路的指令存储器中装入机器码文件 sum\_riscv.hex

| 工程  | 电路仿真     | 窗口       | 帮助                                                                                    |
|-----|----------|----------|---------------------------------------------------------------------------------------|
| 000 | 00a06693 | 00106793 | 00106613 00006713 00f70733 00d786b3 00c787b3 ff5ff06f 00e02023 0000006f 00000000 0000 |
| 010 | 00000000 | 00000000 | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0000 |
| 020 | 00000000 | 00000000 | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0000 |
| 030 | 00000000 | 00000000 | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 0000 |

第五步：设置时钟频率=1KHz，按 Ctrl+K 启动时钟，程序开始执行

第六步：指令存储器停止滚动后，表示程序执行结束

第七步：再按 Ctrl+K 停止时钟



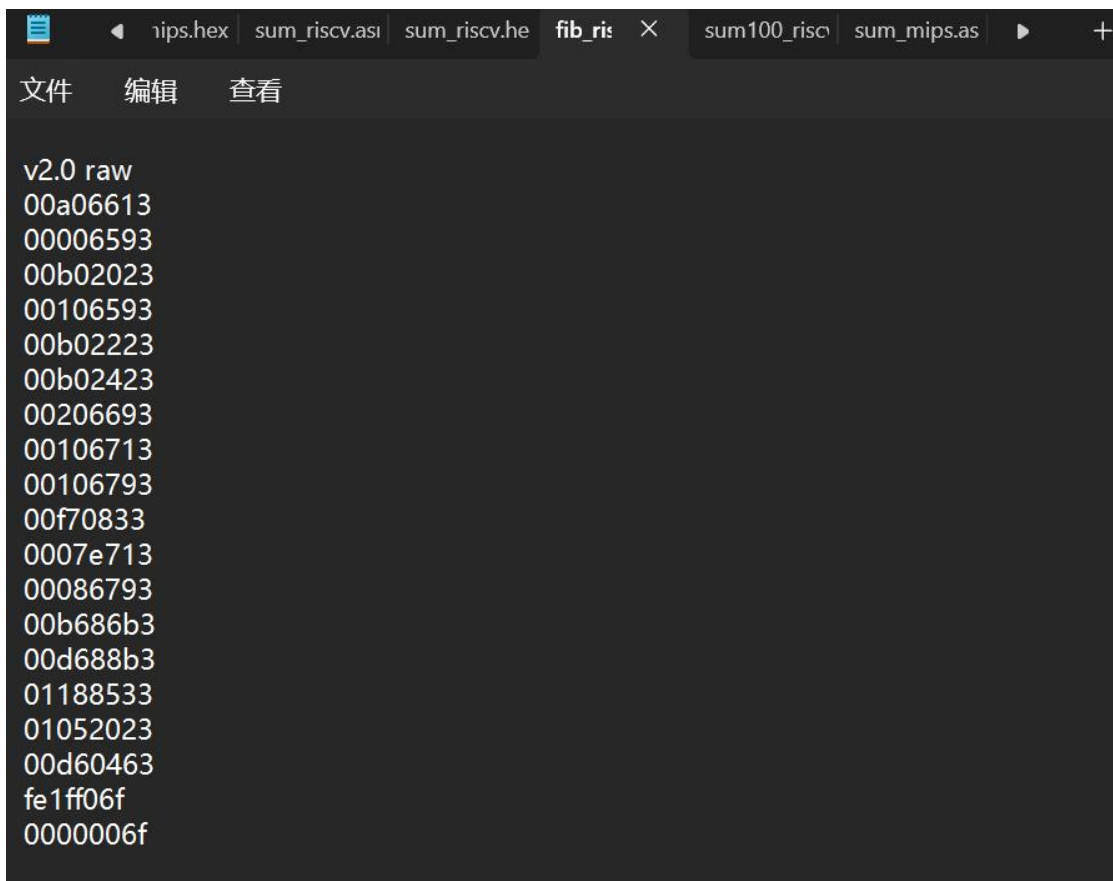
第八步：此时观察数据存储器第 0 号单元的值（累加和，正确值为 37h=55）

| 电路仿真   | 窗口       | 帮助                         |
|--------|----------|----------------------------|
| 000000 | 00000037 | 00000000 00000000 00000000 |
| 000010 | 00000000 | 00000000 00000000 00000000 |
| 000020 | 00000000 | 00000000 00000000 00000000 |
| 000030 | 00000000 | 00000000 00000000 00000000 |
| 000040 | 00000000 | 00000000 00000000 00000000 |
| 000050 | 00000000 | 00000000 00000000 00000000 |
| 000060 | 00000000 | 00000000 00000000 00000000 |

b. 在 Logisim 实现的单周期 RISC-V 处理器上运行计算费波那契数列的程序

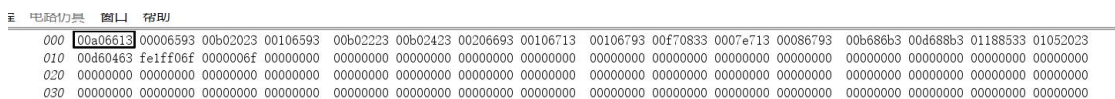
第一步：修改前面导出的机器码文件 fib\_riscv.hex，增加： v2.0 raw





第二步：在 Logisim 中按 Ctrl+R

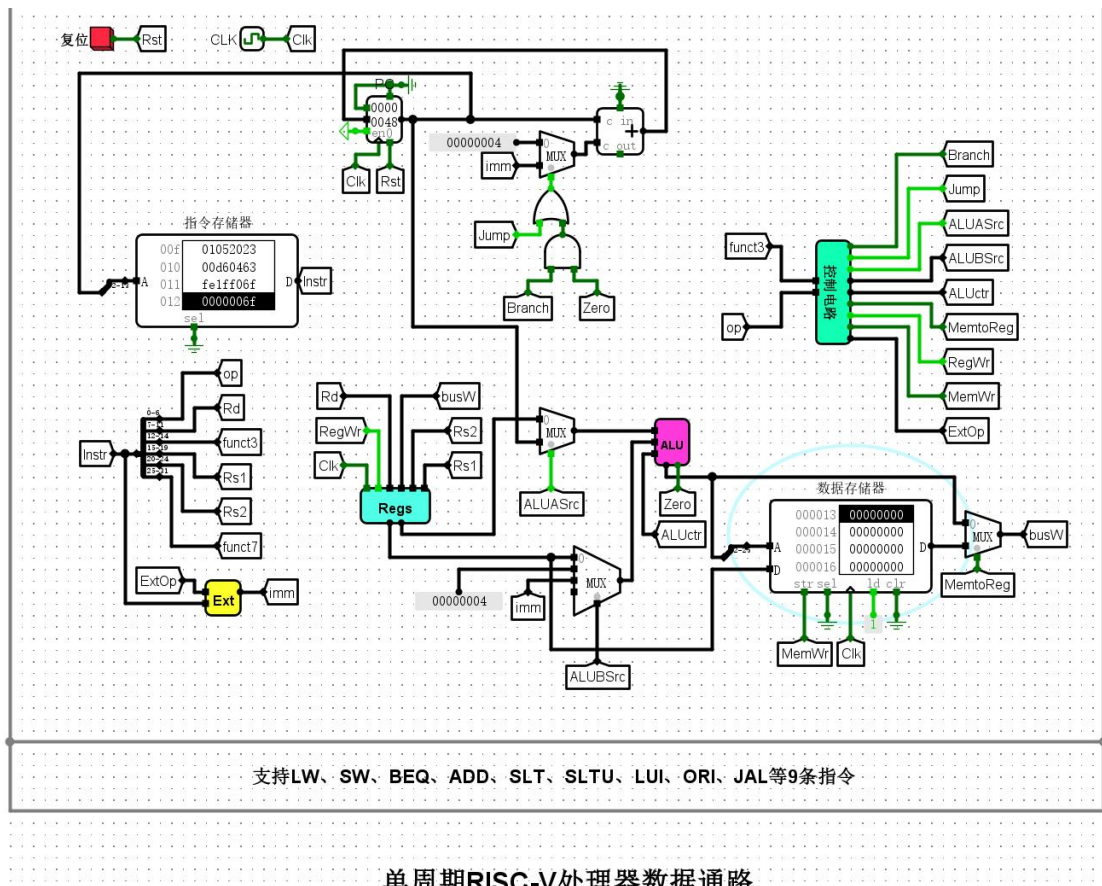
第三步：在指令存储器中装入机器码文件 fib\_riscv.hex



第四步：设置时钟频率=1KHz，按 Ctrl+K 启动时钟，程序开始执行

第五步：指令存储器停止滚动后，表示程序执行结束

第六步：再按 Ctrl+K 停止时钟

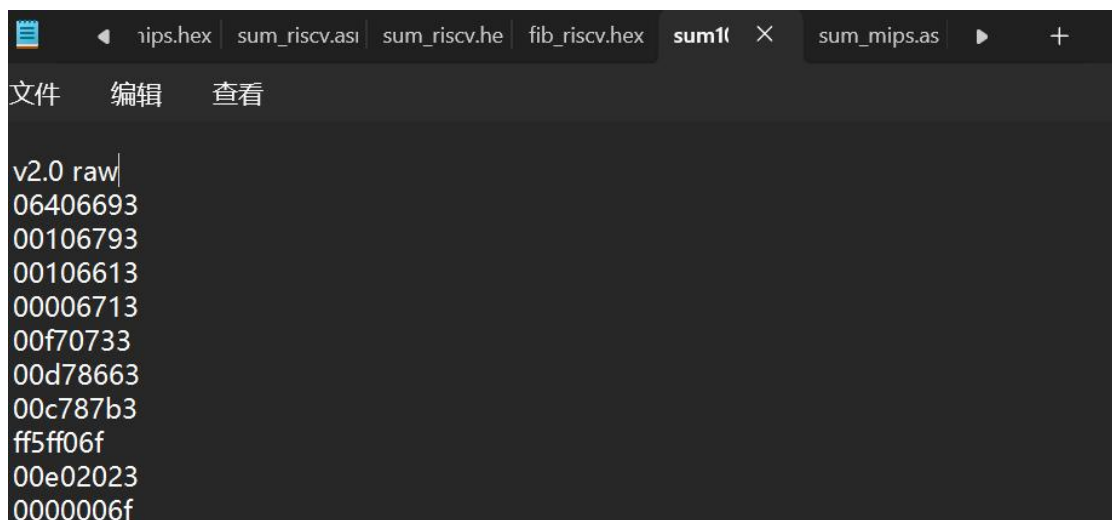


单周期RISC-V处理器数据通路

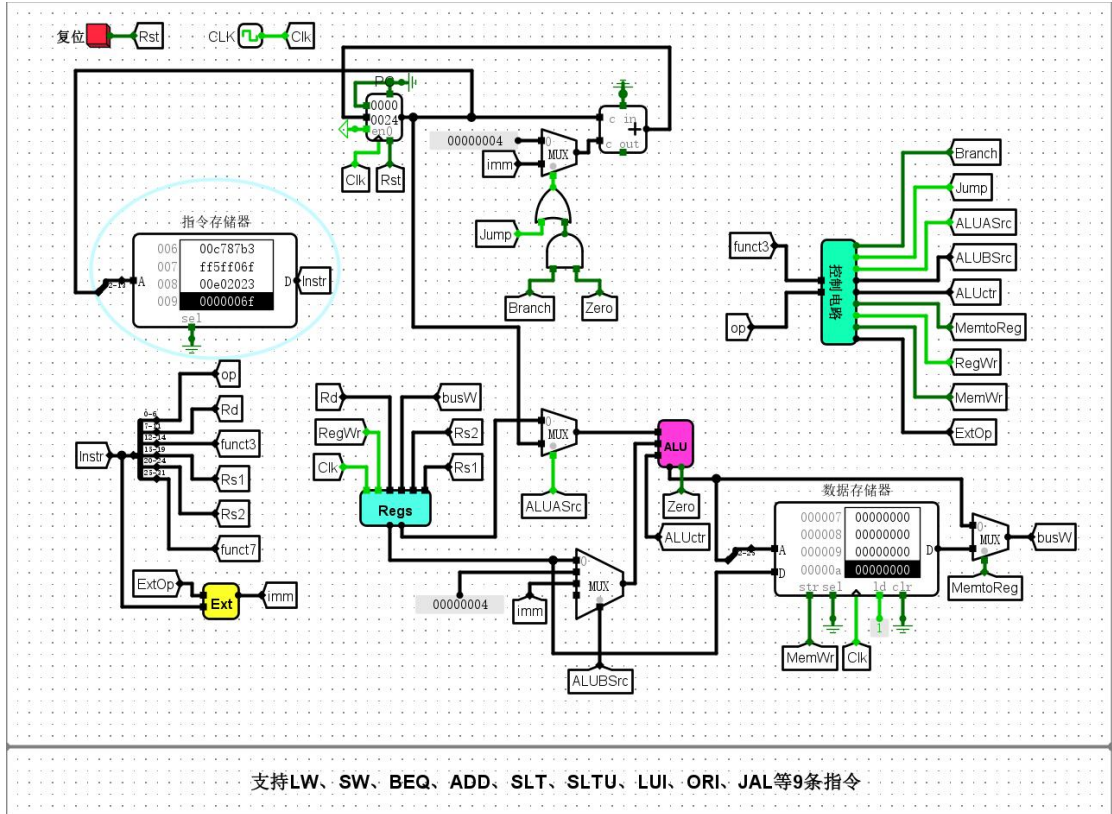
第七步：此时观察数据存储器中的内容（0 号单元开始）

| 工程     | 电路仿真     | 窗口                         | 帮助                                  |
|--------|----------|----------------------------|-------------------------------------|
| 000000 | 00000000 | 00000001 00000001 00000002 | 00000003 00000005 00000008 00000004 |
| 000010 | 00000000 | 00000000 00000000 00000000 | 00000000 00000000 00000000 00000000 |
| 000020 | 00000000 | 00000000 00000000 00000000 | 00000000 00000000 00000000 00000000 |
| 000030 | 00000000 | 00000000 00000000 00000000 | 00000000 00000000 00000000 00000000 |

c. 在 Logisim 实现的单周期 RISC-V 处理器上，运行计算  $1+2+3+\dots+100=5050=13\text{bah}$  的程序（sum100\_riscv.hex），观看数据存储器 0 号单元的值是不是 13bah？



|     |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000 | 06406693 | 00106793 | 00106613 | 00006713 | 00f70733 | 00d78663 | 00c787b3 | ff5ff06f | 00e02023 | 0000006f | 00000000 | 00000000 |
| 010 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 030 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 040 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 050 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |



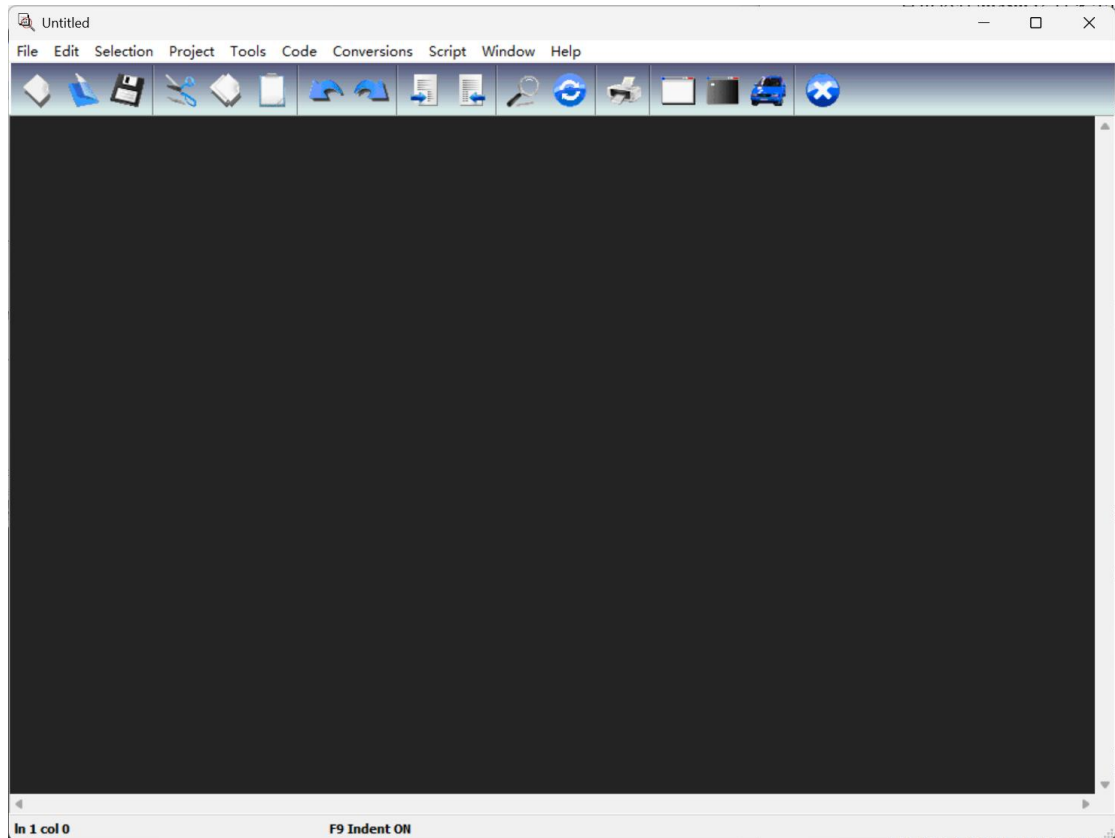
单周期RISC-V处理器数据通路

|        |          |          |          |          |          |    |
|--------|----------|----------|----------|----------|----------|----|
| 000000 | 000013ba | 00000000 | 00000000 | 00000000 | 00000000 | 00 |
| 000010 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00 |
| 000020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00 |
| 000030 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00 |
| 000040 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00 |

## (5) Intel x86 汇编语言程序的运行

### a. Intel x86 汇编工具

点击运行\masm32 目录中的“qeditor.exe”。



b. 运行 Intel x86 汇编语言程序

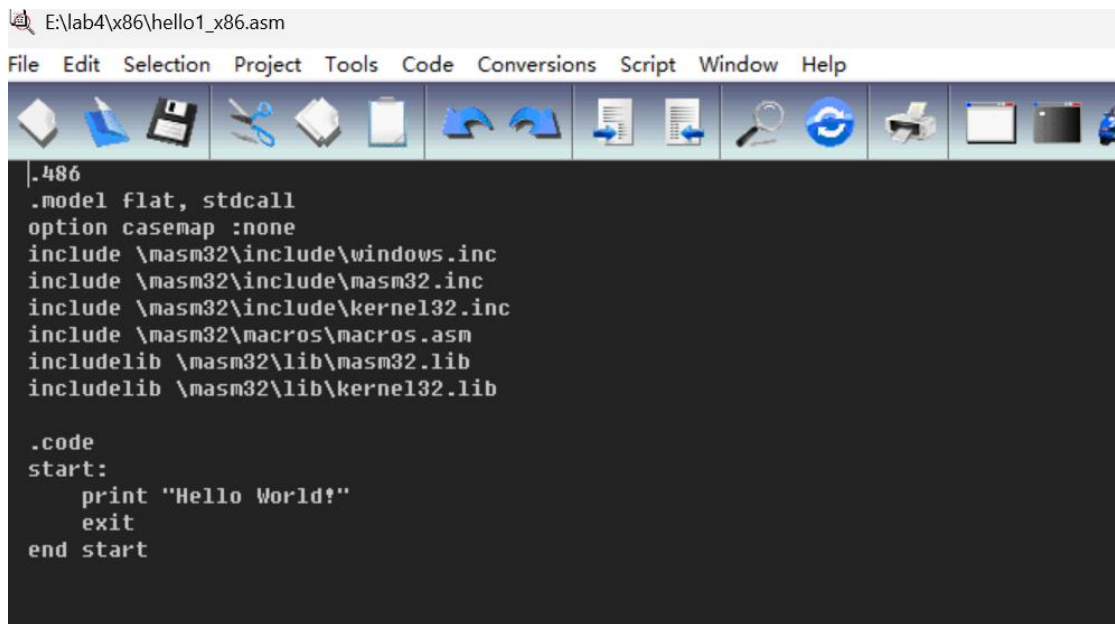
显示“Hello World!”的 Intel x86 汇编语言程序如下（hello1\_x86.asm，位于\x86 目录中）：

```
riscv.as | sum_riscv.he | fib_riscv.hex | sum100_risc | sum_mips.as | hello1 × | ▶ | +
文件 编辑 查看

.486
.model flat, stdcall
option casemap :none
include \masm32\include\windows.inc
include \masm32\include\masm32.inc
include \masm32\include\kernel32.inc
include \masm32\macros\macros.asm
includelib \masm32\lib\masm32.lib
includelib \masm32\lib\kernel32.lib

.code
start:
    print "Hello World!"
    exit
end start
```

第一步：在 masm32 汇编工具中打开源程序“hello1\_x86.asm”

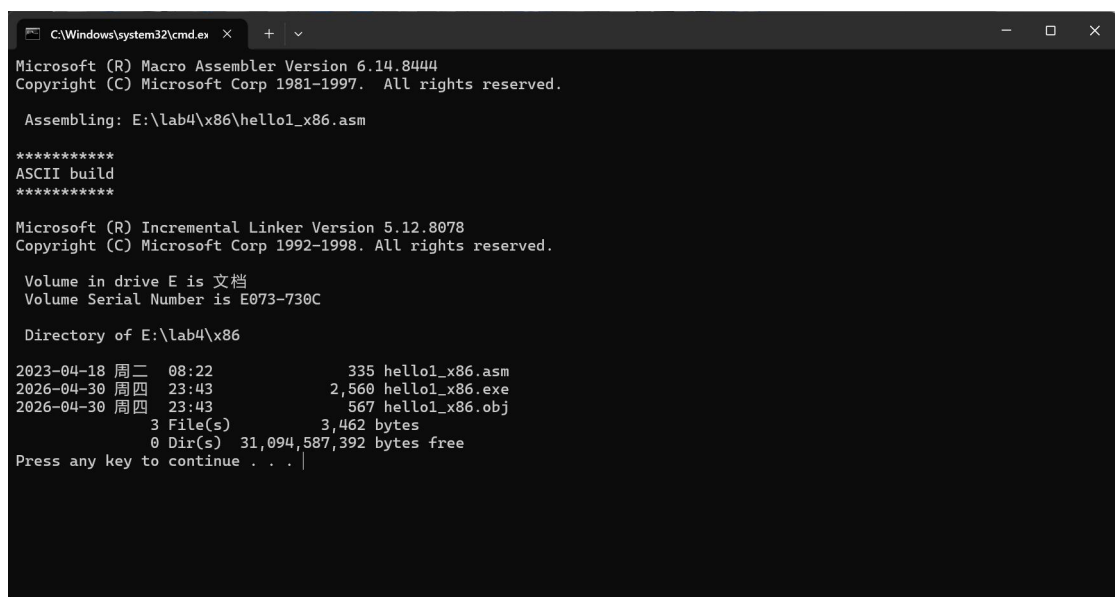


The screenshot shows a Notepad++ window with the file path 'E:\lab4\x86\hello1\_x86.asm'. The menu bar includes File, Edit, Selection, Project, Tools, Code, Conversions, Script, Window, and Help. The toolbar contains various icons for file operations and editing. The code in the editor is as follows:

```
.486
.model flat, stdcall
option casemap :none
include \masm32\include\windows.inc
include \masm32\include\masm32.inc
include \masm32\include\kernel32.inc
include \masm32\macros\macros.asm
includelib \masm32\lib\masm32.lib
includelib \masm32\lib\kernel32.lib

.code
start:
    print "Hello World!"
    exit
end start
```

第二步：汇编



The screenshot shows a Windows Command Prompt window with the title 'C:\Windows\system32\cmd.exe'. The output of the assembly and linking process is as follows:

```
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: E:\lab4\x86\hello1_x86.asm

*****
ASCII build
*****

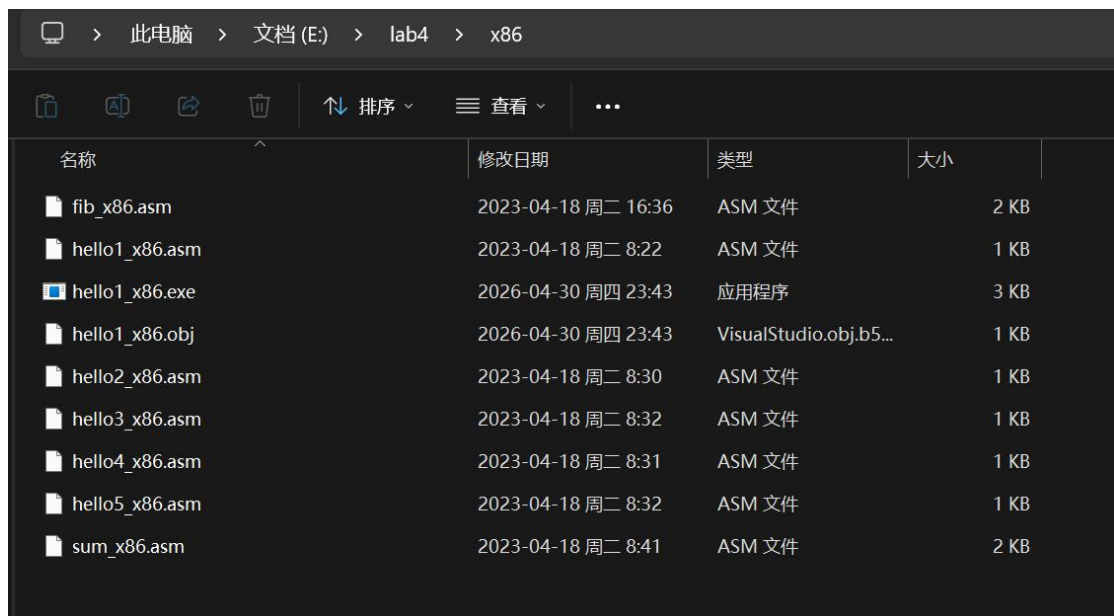
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

Volume in drive E is 文档
Volume Serial Number is E073-730C

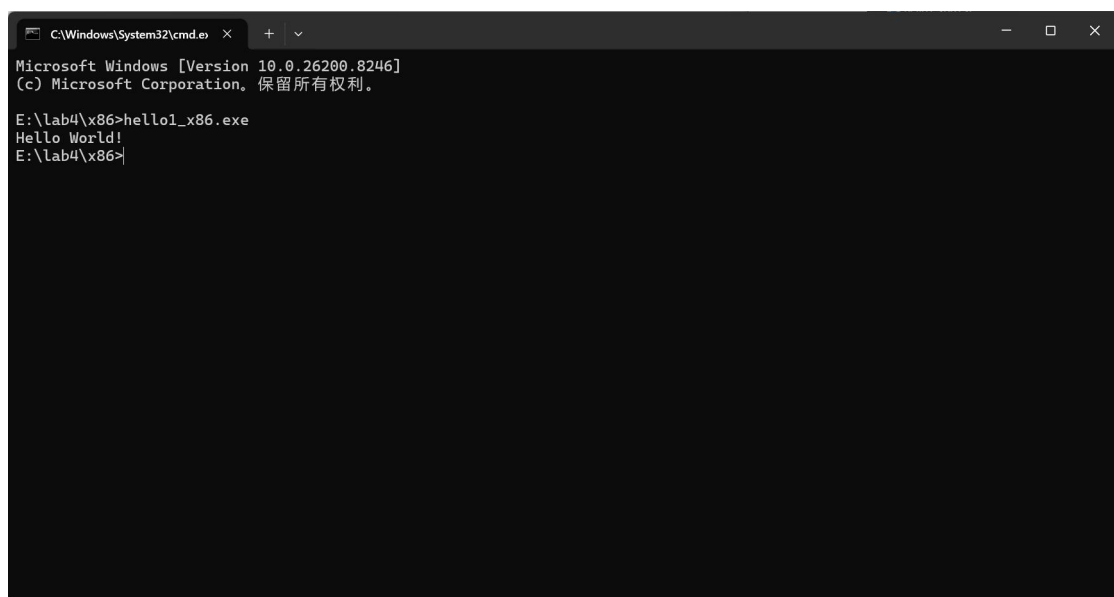
Directory of E:\lab4\x86

2023-04-18 周二 08:22          335 hello1_x86.asm
2026-04-30 周四 23:43        2,560 hello1_x86.exe
2026-04-30 周四 23:43          567 hello1_x86.obj
               3 File(s)      3,462 bytes
               0 Dir(s) 31,094,587,392 bytes free
Press any key to continue . . .
```



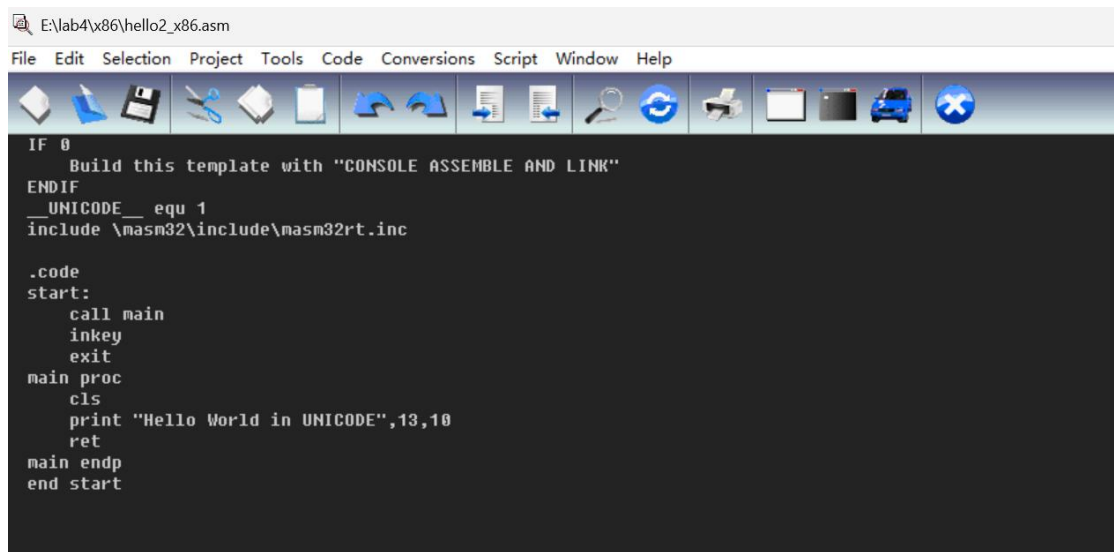


第三步：运行“hello1\_86.exe”



其它版本的 “Hello World”程序

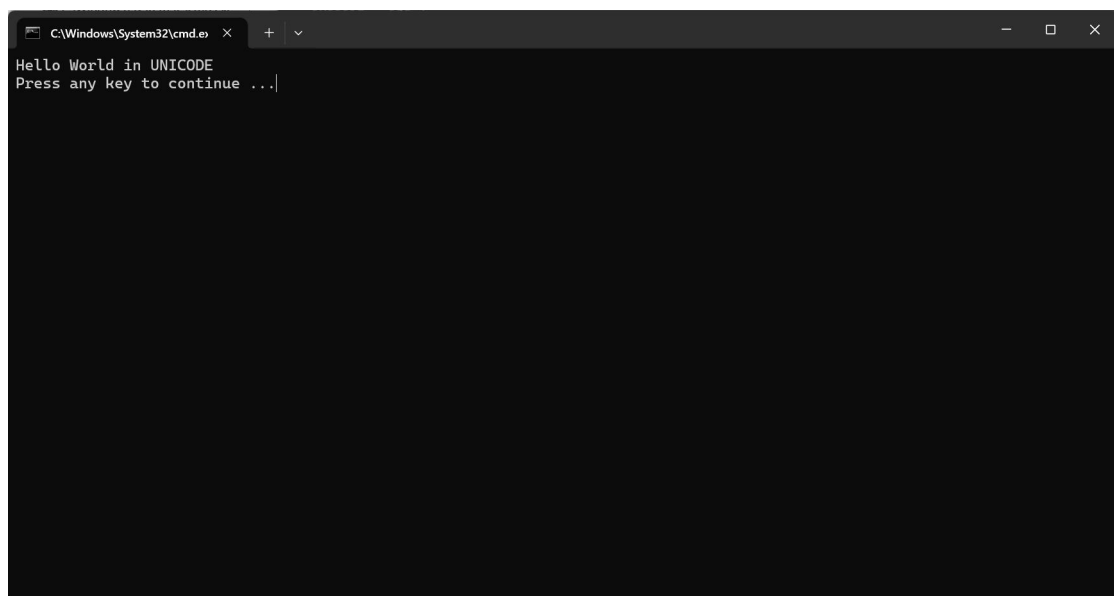
hello2



The screenshot shows a window titled "E:\lab4\x86\hello2\_x86.asm". The menu bar includes File, Edit, Selection, Project, Tools, Code, Conversions, Script, Window, and Help. The toolbar contains icons for file operations (new, open, save, print), editing (undo, redo, copy, paste), and development (assemble, link, debug). The code area contains the following assembly code:

```
IF 0
    Build this template with "CONSOLE ASSEMBLE AND LINK"
ENDIF
__UNICODE__ equ 1
include \masm32\include\masm32rt.inc

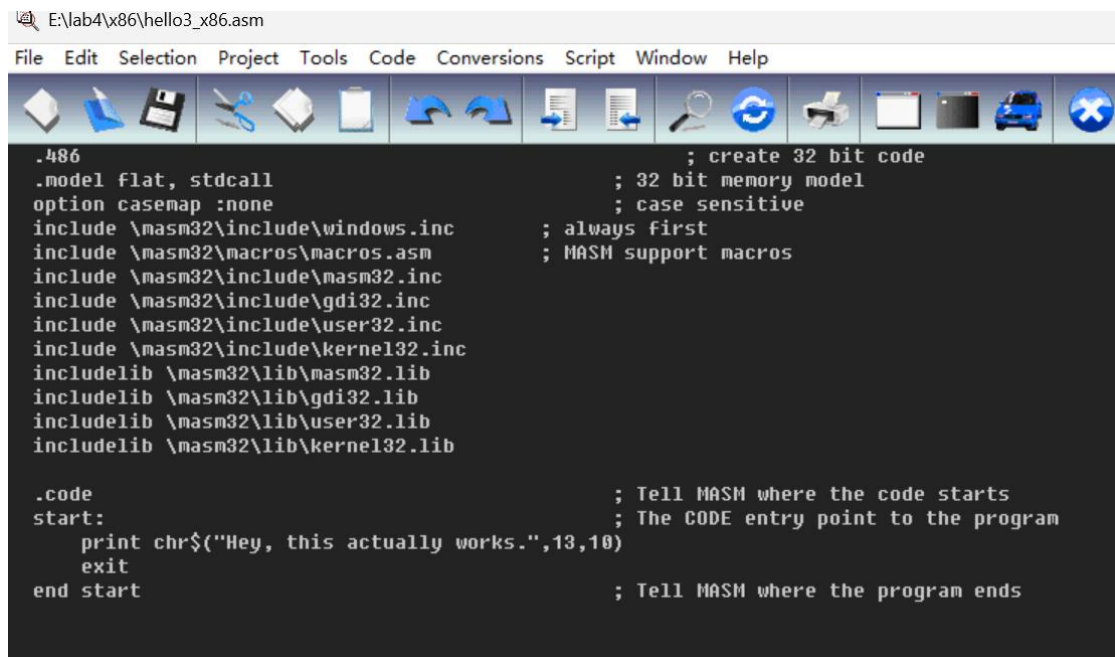
.code
start:
    call main
    inkey
    exit
main proc
    cls
    print "Hello World in UNICODE",13,10
    ret
main endp
end start
```



The screenshot shows a Windows command prompt window with the title bar "C:\Windows\System32\cmd.exe". The window displays the output of the assembly program:

```
Hello World in UNICODE
Press any key to continue ...|
```

hello3



The screenshot shows a MASM32 IDE window titled "E:\lab4\x86\hello3\_x86.asm". The menu bar includes File, Edit, Selection, Project, Tools, Code, Conversions, Script, Window, and Help. The toolbar contains icons for file operations (new, open, save, print), editing (undo, redo, cut, copy, paste), and development (search, build, run, debug). The code editor displays the following assembly code:

```
.486                                     ; create 32 bit code
.model flat, stdcall                   ; 32 bit memory model
option casemap :none                  ; case sensitive
include \masm32\include\windows.inc   ; always first
include \masm32\macros\macros.asm     ; MASM support macros
include \masm32\include\masm32.inc
include \masm32\include\gdi32.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\masm32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.code                                   ; Tell MASM where the code starts
start:                                ; The CODE entry point to the program
    print chr$("Hey, this actually works.",13,10)
    exit
end start                               ; Tell MASM where the program ends
```



The screenshot shows a Windows Command Prompt window titled "C:\Windows\System32\cmd.exe". The output of the program is as follows:

```
Hello World in UNICODE
Press any key to continue ...

E:\lab4\x86>hello3_x86.exe
Hey, this actually works.

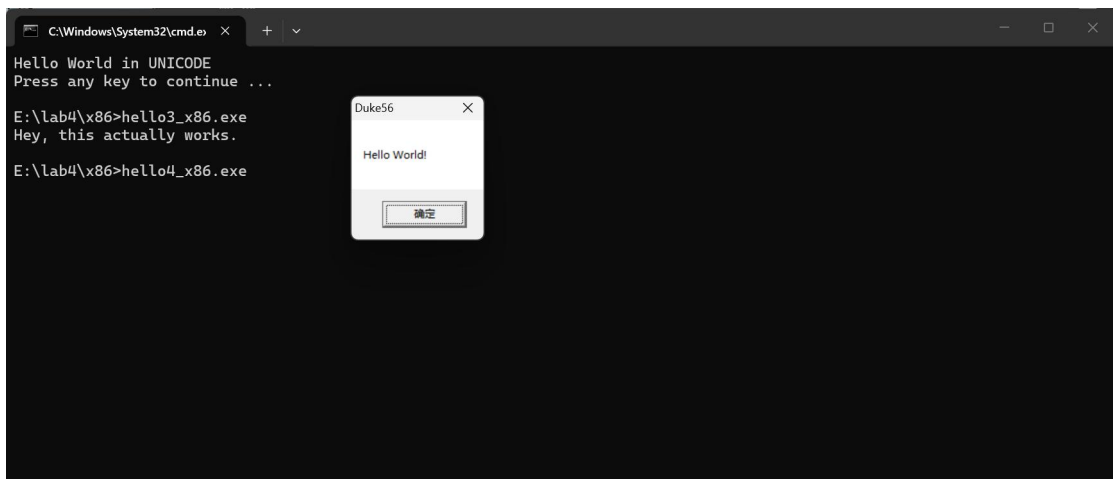
E:\lab4\x86>
```

hello4

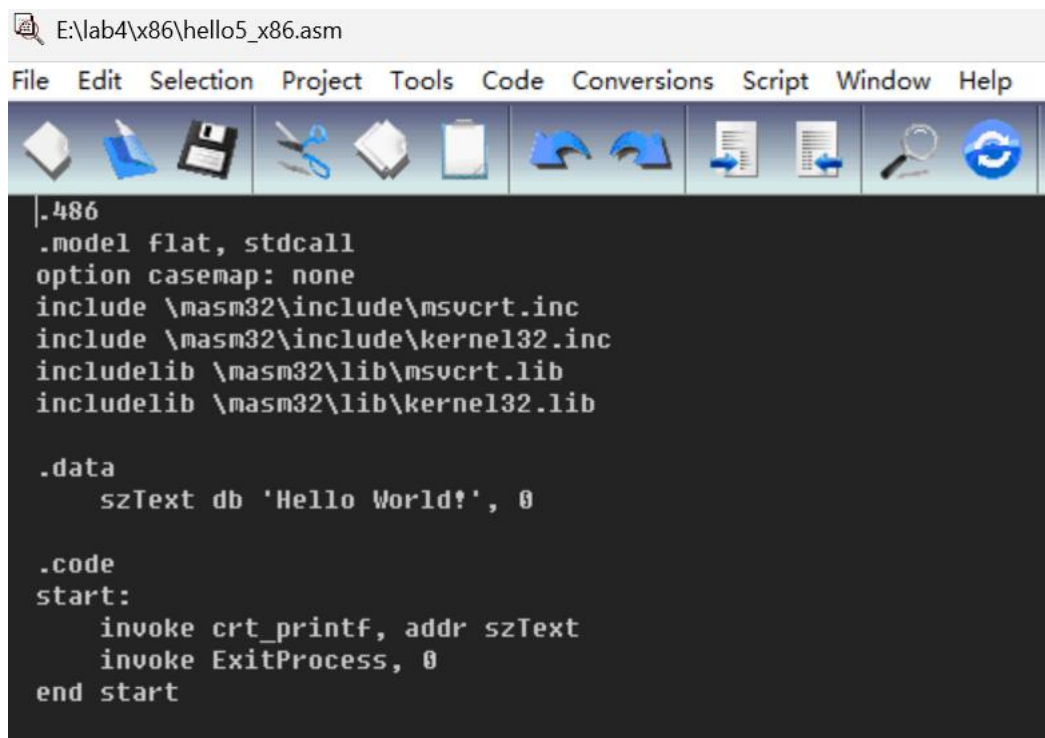
```
E:\lab4\x86\hello4_x86.asm
File Edit Selection Project Tools Code Conversions Script Window Help
[Icons]
.486
.model flat, stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data
    szCaption    db 'Duke56',0
    szText       db 'Hello World!',0

.code
start:
    invoke MessageBox,\
        NULL,\
        offset szText,\
        offset szCaption,\
        MB_OK
    invoke ExitProcess, NULL
end start
```



hello5



The screenshot shows a Notepad++ window with the file path 'E:\lab4\x86\hello5\_x86.asm'. The menu bar includes File, Edit, Selection, Project, Tools, Code, Conversions, Script, Window, and Help. The toolbar contains icons for file operations and editing. The assembly code is as follows:

```
.486
.model flat, stdcall
option casemap: none
include \masm32\include\msvcrt.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\kernel32.lib

.data
    szText db 'Hello World!', 0

.code
start:
    invoke crt_printf, addr szText
    invoke ExitProcess, 0
end start
```



The screenshot shows a Windows Command Prompt window with the title 'C:\Windows\System32\cmd.exe'. It displays the output of three assembly programs:

```
Hello World in UNICODE
Press any key to continue ...

E:\lab4\x86>hello3_x86.exe
Hey, this actually works.

E:\lab4\x86>hello4_x86.exe

E:\lab4\x86>hello5_x86.exe
Hello World!
E:\lab4\x86>
```

c. 运行求累加和的 Intel x86 汇编语言程序

求累加和的 Intel x86 汇编语言程序如下（sum\_x86.asm，位于\x86 目录中）：



```
; x86 (32位) 汇编语言程序      sum x86.asm      求累加和      sum = 1+2+...+n

.486
.model flat, stdcall
option casemap:none
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\masm32.lib
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
n dd ?
sum dd ?
a dd 1
b dd 0

.code
main:
    mov eax, sval(input("Input n= "))    ; 输入n, 送eax
    mov n,eax                            ; eax 保存到 n存储单元中
    mov ecx,n                            ; ecx=n
    mov eax,a                            ; eax=1
    mov ebx,b                            ; ebx=0
sum loop:
    add ebx,eax
    add eax,a
    cmp eax,ecx                          ; 比较eax和ecx
    jg sum stop                          ; 大于则转sum stop
    jmp sum loop
sum stop:
    mov sum,ebx                          ; 累加和结果 (ebx) 保存到sum存储单元中
    print chr$("Sum= ")
    print str$(sum)
    exit
end main
```

第一步：在 masm32 汇编工具中打开源程序“sum\_x86.asm”

```

E:\lab4\x86\sum_x86.asm
File Edit Selection Project Tools Code Conversions Script Window Help
x86 (32位) 汇编语言程序 sum_x86.asm 求累加和 sum = 1+2+...+n

.486
.model flat, stdcall
option casemap:none
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvert.lib
includelib \masm32\lib\masm32.lib
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
n dd ?
sum dd ?
a dd 1
b dd 0

.code
main:
    mov eax, sval(input("Input n= "))    ; 输入n, 送eax
    mov n, eax                          ; eax 保存到 n存储单元中
    mov ecx, n                          ; ecx=n
    mov eax, a                          ; eax=1
    mov ebx, b                          ; ebx=0

sum_loop:
    add ebx, eax
    add eax, a
    cmp eax, ecx                        ; 比较eax和ecx
    jg sum_stop                        ; 大于则转sum_stop

```

第二步：汇编

|                           |                     |                        |      |  |
|---------------------------|---------------------|------------------------|------|--|
| 此电脑 > 文档(E:) > lab4 > x86 |                     |                        |      |  |
| 名称                        | 修改日期                | 类型                     | 大小   |  |
| hello1_x86.obj            | 2026-04-30 周四 23:43 | VisualStudio.obj.b5... | 1 KB |  |
| hello2_x86.asm            | 2023-04-18 周二 8:30  | ASM 文件                 | 1 KB |  |
| hello3_x86.asm            | 2023-04-18 周二 8:32  | ASM 文件                 | 1 KB |  |
| hello4_x86.asm            | 2023-04-18 周二 8:31  | ASM 文件                 | 1 KB |  |
| hello5_x86.asm            | 2023-04-18 周二 8:32  | ASM 文件                 | 1 KB |  |
| sum_x86.asm               | 2023-04-18 周二 8:41  | ASM 文件                 | 2 KB |  |
| hello2_x86.obj            | 2026-04-30 周四 23:47 | VisualStudio.obj.b5... | 2 KB |  |
| hello2_x86.exe            | 2026-04-30 周四 23:47 | 应用程序                   | 3 KB |  |
| hello3_x86.obj            | 2026-04-30 周四 23:48 | VisualStudio.obj.b5... | 1 KB |  |
| hello3_x86.exe            | 2026-04-30 周四 23:48 | 应用程序                   | 3 KB |  |
| hello4_x86.obj            | 2026-04-30 周四 23:49 | VisualStudio.obj.b5... | 1 KB |  |
| hello4_x86.exe            | 2026-04-30 周四 23:49 | 应用程序                   | 3 KB |  |
| hello5_x86.obj            | 2026-04-30 周四 23:50 | VisualStudio.obj.b5... | 1 KB |  |
| hello5_x86.exe            | 2026-04-30 周四 23:50 | 应用程序                   | 3 KB |  |
| sum_x86.obj               | 2026-04-30 周四 23:53 | VisualStudio.obj.b5... | 2 KB |  |
| sum_x86.exe               | 2026-04-30 周四 23:53 | 应用程序                   | 3 KB |  |

第三步：运行“sum\_x86.exe”



```
C:\Windows\System32\cmd.exe
Hello World in UNICODE
Press any key to continue ...

E:\lab4\x86>hello3_x86.exe
Hey, this actually works.

E:\lab4\x86>hello4_x86.exe

E:\lab4\x86>hello5_x86.exe
Hello World!
E:\lab4\x86>sum_x86.exe
Input n= 10
Sum= 55
E:\lab4\x86>
```

d. 运行计算费波那契数列的 Intel x86 汇编语言程序

计算费波那契数列的 Intel x86 汇编语言程序如下（fib\_x86.asm，位于\x86 目录中）：

```
100_risc | sum_mips.as | hello1_x86.a: | hello2_x86.a: | sum_x86.asr | fib_x86 X
文件 编辑 查看

.486
.model flat, stdcall

option casemap:none

includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\masm32.lib

include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
    n dd 0
    a dd 0
    b dd 1
    i dd 1
    t dd 0

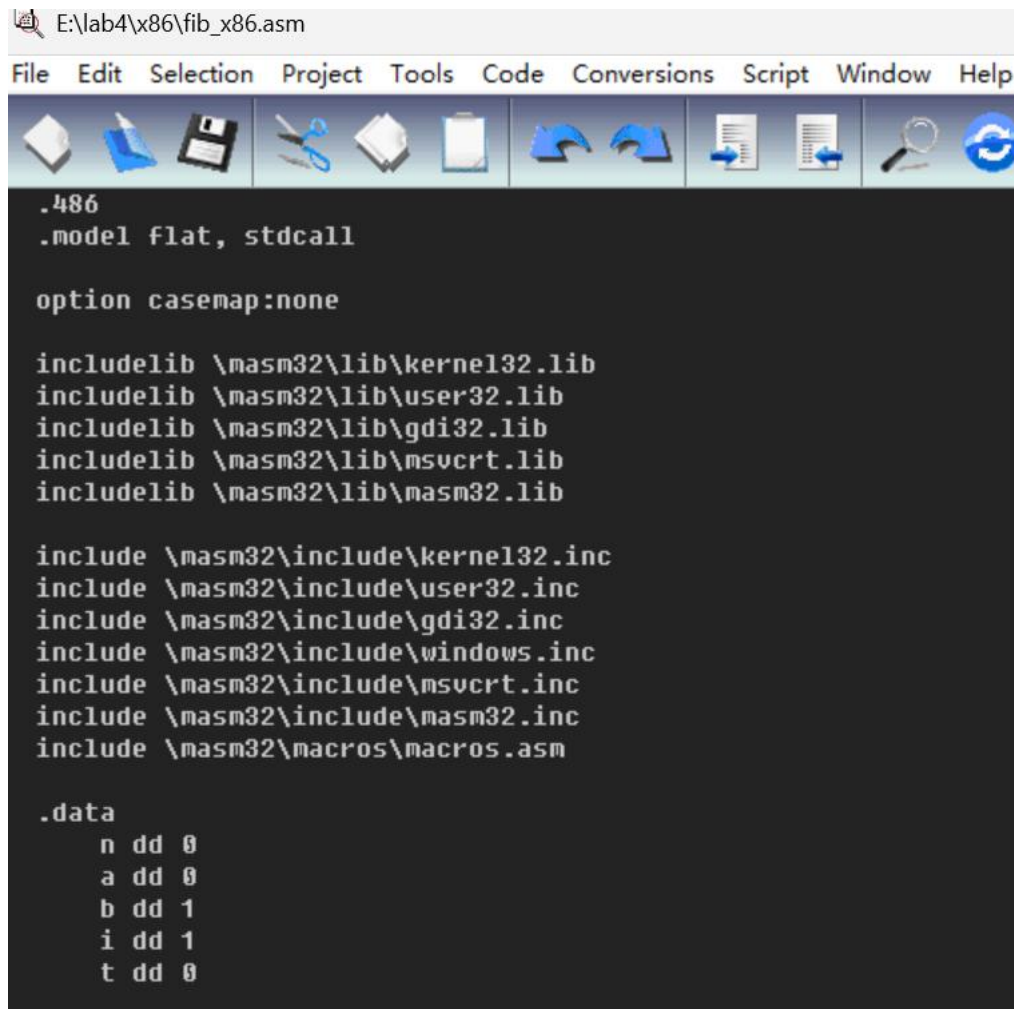
.code
start:
    mov eax, sval(input("Enter a number : "))
    mov n,eax

    print chr$("Fibonacci number")
    print str$(0)
    print chr$(" is ")
    print str$(a)
    print chr$(" ",13,10)

    print chr$("Fibonacci number")
    print str$(1)
    print chr$(" is ")
    print str$(b)
    print chr$(" ",13,10)

    mov ecx,n
    mov i,ecx
    dec i
    .while i
        mov eax,b
        mov t,eax
        mov eax,b
        add eax,a
        mov b,eax
        print chr$("Fibonacci number")
        mov ecx,n
        sub ecx,i
        inc ecx
        print str$(ecx)
        print chr$(" is ")
        print str$(b)
        print chr$(" ",13,10)
        mov eax,t
        mov a,eax
        dec i
    .endw
    ret
ret
```

第一步：在 masm32 汇编工具中打开源程序“fib\_x86.asm”

The image shows a screenshot of the MASM32 IDE. The title bar at the top reads 'E:\lab4\x86\fib\_x86.asm'. Below the title bar is a menu bar with the following items: File, Edit, Selection, Project, Tools, Code, Conversions, Script, Window, and Help. Underneath the menu bar is a toolbar containing various icons for file operations (like opening, saving, deleting), editing (like undo, redo, copy, paste), and development (like running, debugging). The main area of the IDE is a dark background with white text showing the assembly code for 'fib\_x86.asm'. The code includes directives for the 486 model, standard calling convention, and no case mapping. It also includes several libraries and header files from the MASM32 SDK. Finally, it defines a data segment with five variables: n, a, b, i, and t, each initialized with a specific value.

```
.486
.model flat, stdcall

option casemap:none

includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\masm32.lib

include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
    n dd 0
    a dd 0
    b dd 1
    i dd 1
    t dd 0
```

第二步：汇编



此电脑 > 文档 (E:) > lab4 > x86

| 名称             | 修改日期                | 类型                     | 大小   |
|----------------|---------------------|------------------------|------|
| hello3_x86.asm | 2023-04-18 周二 8:32  | ASM 文件                 | 1 KB |
| hello4_x86.asm | 2023-04-18 周二 8:31  | ASM 文件                 | 1 KB |
| hello5_x86.asm | 2023-04-18 周二 8:32  | ASM 文件                 | 1 KB |
| sum_x86.asm    | 2023-04-18 周二 8:41  | ASM 文件                 | 2 KB |
| hello2_x86.obj | 2026-04-30 周四 23:47 | VisualStudio.obj.b5... | 2 KB |
| hello2_x86.exe | 2026-04-30 周四 23:47 | 应用程序                   | 3 KB |
| hello3_x86.obj | 2026-04-30 周四 23:48 | VisualStudio.obj.b5... | 1 KB |
| hello3_x86.exe | 2026-04-30 周四 23:48 | 应用程序                   | 3 KB |
| hello4_x86.obj | 2026-04-30 周四 23:49 | VisualStudio.obj.b5... | 1 KB |
| hello4_x86.exe | 2026-04-30 周四 23:49 | 应用程序                   | 3 KB |
| hello5_x86.obj | 2026-04-30 周四 23:50 | VisualStudio.obj.b5... | 1 KB |
| hello5_x86.exe | 2026-04-30 周四 23:50 | 应用程序                   | 3 KB |
| sum_x86.obj    | 2026-04-30 周四 23:53 | VisualStudio.obj.b5... | 2 KB |
| sum_x86.exe    | 2026-04-30 周四 23:53 | 应用程序                   | 3 KB |
| fib_x86.obj    | 2026-04-30 周四 23:58 | VisualStudio.obj.b5... | 3 KB |
| fib_x86.exe    | 2026-04-30 周四 23:58 | 应用程序                   | 3 KB |

```

C:\Windows\System32\cmd.exe
Hello World in UNICODE
Press any key to continue ...

E:\lab4\x86>hello3_x86.exe
Hey, this actually works.

E:\lab4\x86>hello4_x86.exe

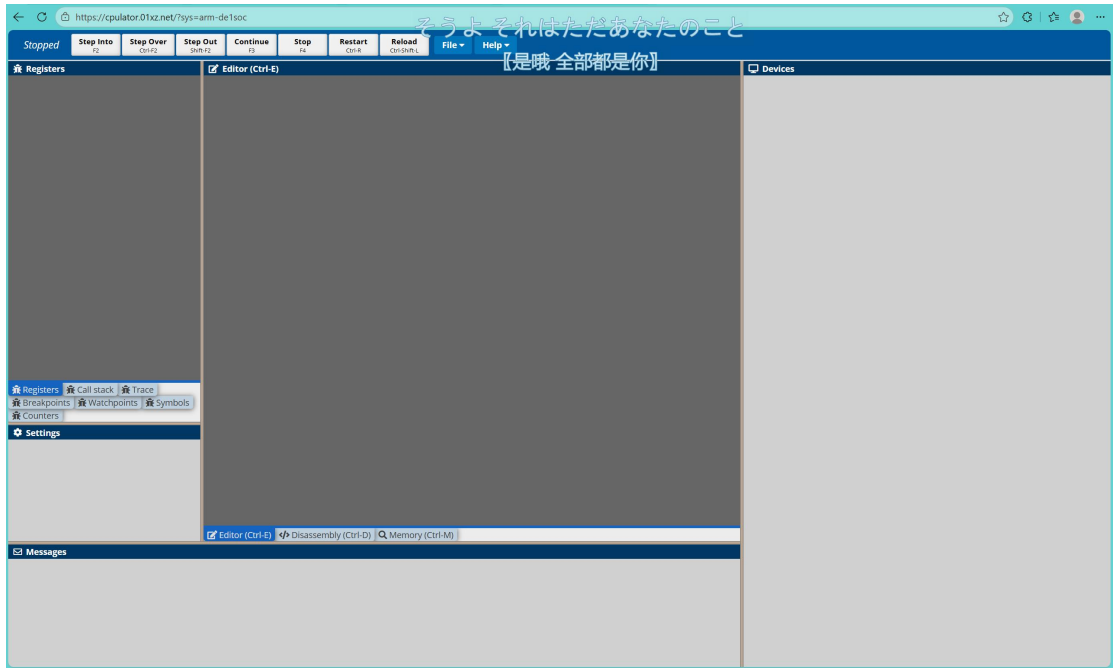
E:\lab4\x86>hello5_x86.exe
Hello World!
E:\lab4\x86>sum_x86.exe
Input n= 10
Sum= 55
E:\lab4\x86>fib_x86.exe
Enter a number : 10
Fibonacci_number0 is 0
Fibonacci_number1 is 1
Fibonacci_number2 is 1
Fibonacci_number3 is 2
Fibonacci_number4 is 3
Fibonacci_number5 is 5
Fibonacci_number6 is 8
Fibonacci_number7 is 13
Fibonacci_number8 is 21
Fibonacci_number9 is 34
Fibonacci_number10 is 55

```

## (6) ARMv7 汇编语言程序的运行

### a. ARMv7 汇编工具

在浏览器中打开 ARMv7 汇编工具：<https://cpulator.01xz.net/?sys=arm-de1soc>



b. 在 ARMv7 汇编工具中运行求累加和程序

求累加和的 ARMv7 汇编语言程序如下（sum\_armv7.s，位于\armv7 目录中）：

```
3_x86.a: hello4_x86.a: hello5_x86.a: sum_x86.asr fib_armv7.s sum_i X +
文件 编辑 查看

#ARMv7 汇编语言程序      求累加和      sum=1+2+3+.....+n      sum armv7.s
# 累加和个数n放在[0x100]单元中      可以改变
# 累加和结果sum放在[0x104]单元中

.global start
start:
.org 0
    ldr r4,num           // r4=0x100
    ldr r5,result        // r5=0x104
    mov r0,#100          // r0=100
    str r0,[r4]          // 将r0保存到[0x100]单元中
    ldr r3,[r4]          // 从[0x100]单元中读取数据，送r3
    mov r0,#1
    mov r1,#0
sum:
    add r1,r0
    add r0,#1
    cmp r0,r3
    bgt end              // 如果r0 > r3 则转 end
    b sum                // 转 sum
end:
    str r1,[r5]          // 将r1 保存到 [0x104]单元中
stop:
    b stop               // 转 stop

.org 0x100
num:
    .word 0

result:
    .word 0
```

第一步：打开源程序（sum\_armv7.s）

```

1 #ARMv7 汇编语言程序      求累加和      sum=1+2+3+.....+n      sum_armv7.s
2 # 累加和个数n放在[0x100]单元中      可以改变
3 # 累加和结果sum放在[0x104]单元中
4
5 .global _start
6 _start:
7 .org 0
8     ldr r4,=num                // r4=0x100
9     ldr r5,=result            // r5=0x104
10    mov r0,#100                // r0=100
11    str r0,[r4]                // 将r0保存到[0x100]单元中
12    ldr r3,[r4]                // 从[0x100]单元中读取数据,送r3
13    mov r0,#1
14    mov r1,#0
15 sum:
16     add r1,r0
17     add r0,#1
18     cmp r0,r3
19     bgt end                    // 如果r0 > r3 则转 end
20     b sum                      // 转 sum
21 end:
22     str r1,[r5]                // 将r1 保存到 [0x104]单元中
23 stop:
24     b stop                      // 转 stop
25
26 .org 0x100
27 num:
28     .word 0
29
30 result:
31     .word 0
32
33

```

第二步：汇编

```

CPUlator has started with system ARMv7 DE1-SoC containing a ARMv7 processor.
Compiling...
Code and data loaded from ELF executable into memory. Total size is 272 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmibltQ.s.o work/asmibltQ.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmibltQ.s.elf work/asmibltQ.s.o
Compile succeeded.

```

第三步：连续执行程序

| Disassembly (Ctrl-D)                                                             |          |                          |
|----------------------------------------------------------------------------------|----------|--------------------------|
| Go to address, label, or register: <input type="text" value="00000000"/> Refresh |          |                          |
| Address                                                                          | Opcode   | Disassembly              |
| 00000000                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000e0                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000e4                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000e8                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000ec                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000f0                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000f4                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000f8                                                                         | 00000000 | andeq r0, r0, r0         |
| 000000fc                                                                         | 00000000 | andeq r0, r0, r0         |
| num:                                                                             |          |                          |
| 00000100                                                                         | 00000064 | andeq r0, r0, r4, RRX    |
| result:                                                                          |          |                          |
| 00000104                                                                         | 000013ba | strheq r1, [r0], -r10    |
| 8 ldr r4,=num // r4=0x100                                                        |          |                          |
| 00000108                                                                         | 00000100 | andeq r0, r0, r0, LSL #2 |
| 9 ldr r5,=result // r5=0x104                                                     |          |                          |
| 0000010c                                                                         | 00000104 | andeq r0, r0, r4, LSL #2 |
| _end:                                                                            |          |                          |
| 00000110                                                                         | aaaaaaaa | bge 0xfeaaabc0           |
| 00000114                                                                         | aaaaaaaa | bge 0xfeaaabc4           |
| 00000118                                                                         | aaaaaaaa | bge 0xfeaaabc8           |
| 0000011c                                                                         | aaaaaaaa | bge 0xfeaaabc0           |

| Address  | Memory contents and ASCII |           |          |           |                     |
|----------|---------------------------|-----------|----------|-----------|---------------------|
| 00000000 | e59f4100                  | e59f5100  | e3a00064 | e5840000  | •A••••Q•••d•••••••• |
| 00000010 | e5943000                  | e3a00001  | e3a01000 | e0811000  | •0••••••••••••••••  |
| 00000020 | e2800001                  | e1500003  | ca000000 | eaafffffa | ••••••••P••••••••   |
| 00000030 | e5851000                  | eaafffffe | 00000000 | 00000000  | ••••••••••••••••    |
| 00000040 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••••••••••••••    |
| 00000050 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••••~••••••~••••  |
| 00000060 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 00000070 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 00000080 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 00000090 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000a0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000b0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000c0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000d0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000e0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 000000f0 | 00000000                  | 00000000  | 00000000 | 00000000  | ••••~••••~••••~•••  |
| 00000100 | 00000064                  | 000013ba  | 00000100 | 00000104  | d••••••••••••••••   |
| 00000110 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000120 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000130 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000140 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000150 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000160 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••••~•••  |
| 00000170 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 00000180 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 00000190 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001a0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001b0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001c0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001d0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001e0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |
| 000001f0 | aaaaaaaa                  | aaaaaaaa  | aaaaaaaa | aaaaaaaa  | ••••~••••~••~•~•••  |



#### 第四步：单步执行程序

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

| Address  | Opcode   | Disassembly                                                             |
|----------|----------|-------------------------------------------------------------------------|
| 00000000 | e59f4100 | <b>_start:</b><br>ldr r4,=num // r4=0x100<br>ldr r4, [pc, #256] ; 0x108 |
| 00000004 | e59f5100 | ldr r5,=result // r5=0x104<br>ldr r5, [pc, #256] ; 0x10c                |
| 00000008 | e3a00064 | mov r0, #100 // r0=100<br>mov r0, #100 ; 0x64                           |
| 0000000c | e5840000 | str r0, [r4] // 将r0保存到[0x100]单元中<br>str r0, [r4]                        |
| 00000010 | e5943000 | ldr r3, [r4] // 从[0x100]单元中读取数据, 送r3                                    |
| 00000014 | e3a00001 | mov r0, #1 ; 0x1                                                        |
| 00000018 | e3a01000 | mov r1, #0 ; 0x0                                                        |
| 0000001c | e0811000 | sum:<br>add r1, r0<br>sum:<br>add r1, r1, r0                            |
| 00000020 | e2800001 | add r0, #1                                                              |
| 00000024 | e1500003 | add r0, r0, #1 ; 0x1<br>cmp r0, r3                                      |
| 00000028 | ca000000 | bgt end // 如果r0 > r3 则转 end<br>bgt 0x30 (0x30: end)                     |
| 0000002c | eaffffff | b sum // 转 sum<br>b 0x1c (0x1c: sum)                                    |
| 00000030 | e5851000 | end:<br>str r1, [r5] // 将r1 保存到 [0x104]单元中<br>end:<br>str r1, [r5]      |
|          |          | stop:<br>b stop // 转 stop<br>stop:                                      |

Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)



Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

| Address  | Opcode   | Disassembly                                                |
|----------|----------|------------------------------------------------------------|
|          |          | 8 ldr r4,=num // r4=0x100                                  |
| 00000000 | e59f4100 | _start:<br>ldr r4, [pc, #256] ; 0x108                      |
| 00000004 | e59f5100 | 9 ldr r5,=result // r5=0x104<br>ldr r5, [pc, #256] ; 0x10c |
| 00000008 | e3a00064 | 10 mov r0,#100 // r0=100<br>mov r0, #100 ; 0x64            |
| 0000000c | e5840000 | 11 str r0,[r4] // 将r0保存到[0x100]单元中<br>str r0, [r4]         |
| 00000010 | e5943000 | 12 ldr r3,[r4] // 从[0x100]单元中读取数据, 送r3<br>ldr r3, [r4]     |
| 00000014 | e3a00001 | 13 mov r0, #1 ; 0x1                                        |
| 00000018 | e3a01000 | 14 mov r1, #0 ; 0x0                                        |
|          |          | 15 sum:<br>16 add r1,r0                                    |
| 0000001c | e0811000 | sum:<br>add r1, r1, r0                                     |
| 00000020 | e2800001 | 17 add r0,#1                                               |
| 00000024 | e1500003 | 18 add r0, r0, #1 ; 0x1<br>cmp r0, r3                      |
| 00000028 | ca000000 | 19 bgt end // 如果r0 > r3 则转 end<br>bgt 0x30 (0x30: end)     |
| 0000002c | eaaffffa | 20 b sum // 转 sum<br>b 0x1c (0x1c: sum)                    |
|          |          | 21 end:<br>22 str r1,[r5] // 将r1 保存到 [0x104]单元中            |
| 00000030 | e5851000 | end:<br>str r1, [r5]                                       |
|          |          | 23 stop:<br>24 b stop // 转 stop                            |
|          |          | stop:                                                      |

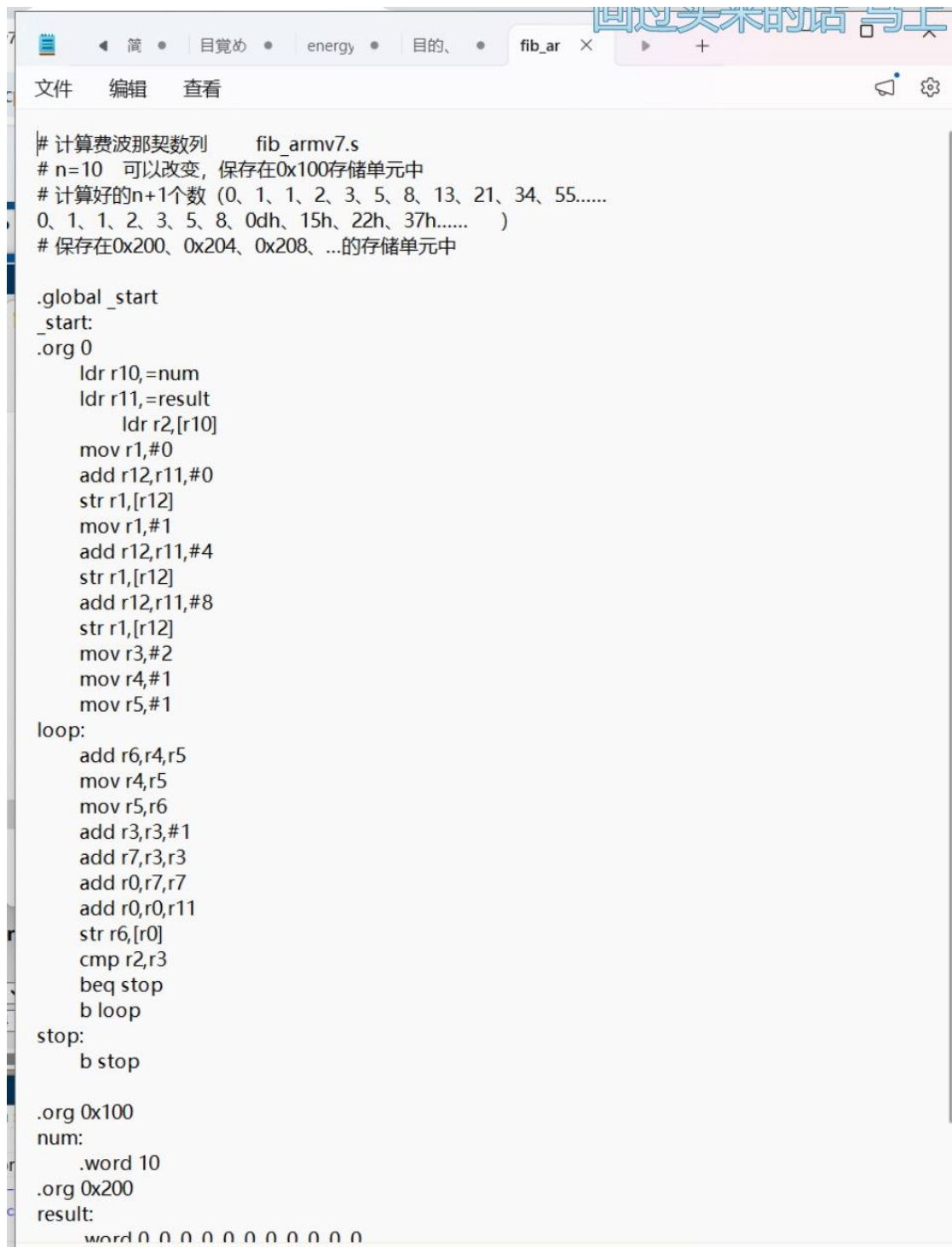
Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)

| Disassembly (Ctrl-D)                                |          |                                        |
|-----------------------------------------------------|----------|----------------------------------------|
| Go to address, label, or register: 00000000 Refresh |          |                                        |
| Address                                             | Opcode   | Disassembly                            |
|                                                     |          | 8 ldr r4,num // r4=0x100               |
| 00000000                                            | e59f4100 | _start:<br>ldr r4,[pc, #256] ; 0x108   |
| 00000004                                            | e59f5100 | ldr r5,result // r5=0x104              |
|                                                     |          | ldr r5,[pc, #256] ; 0x10c              |
| 00000008                                            | e3a00064 | 10 mov r0,#100 // r0=100               |
|                                                     |          | mov r0, #100 ; 0x64                    |
| 0000000c                                            | e5840000 | 11 str r0,[r4] // 将r0保存到[0x100]单元中     |
|                                                     |          | str r0, [r4]                           |
| 00000010                                            | e5943000 | 12 ldr r3,[r4] // 从[0x100]单元中读取数据, 送r3 |
| 00000014                                            | e3a00001 | ldr r3,[r4]                            |
| 00000018                                            | e3a01000 | 13 mov r0, #1 ; 0x1                    |
|                                                     |          | 14 mov r1, #0 ; 0x0                    |
|                                                     |          | 15 sum:                                |
|                                                     |          | 16 add r1,r0                           |
| 0000001c                                            | e0811000 | sum:<br>add r1, r1, r0                 |
|                                                     |          | 17 add r0,#1                           |
| 00000020                                            | e2800001 | add r0, r0, #1 ; 0x1                   |
| 00000024                                            | e1500003 | 18 cmp r0, r3                          |
|                                                     |          | 19 bgt end // 如果r0 > r3 则转 end         |
| 00000028                                            | ca000000 | bgt 0x30 (0x30: end)                   |
|                                                     |          | 20 b sum // 转 sum                      |
| 0000002c                                            | eaaffffa | b 0x1c (0x1c: sum)                     |
|                                                     |          | 21 end:                                |
|                                                     |          | 22 str r1,[r5] // 将r1 保存到 [0x104]单元中   |
| 00000030                                            | e5851000 | end:<br>str r1, [r5]                   |
|                                                     |          | 23 stop:                               |
|                                                     |          | 24 b stop // 转 stop                    |
|                                                     |          | stop:                                  |

后续略，与连续执行结果一致

#### c. 在 ARMv7 汇编工具中运行计算费波那契数列程序

计算费波那契数列的 ARMv7 汇编语言程序如下（fib\_armv7.s，位于\armv7 目录中）：



```
# 计算费波那契数列      fib_armv7.s
# n=10 可以改变, 保存在0x100存储单元中
# 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、55.....
# 0、1、1、2、3、5、8、0dh、15h、22h、37h..... )
# 保存在0x200、0x204、0x208、...的存储单元中

.global _start
_start:
.org 0
    ldr r10,=num
    ldr r11,=result
    ldr r2,[r10]
    mov r1,#0
    add r12,r11,#0
    str r1,[r12]
    mov r1,#1
    add r12,r11,#4
    str r1,[r12]
    add r12,r11,#8
    str r1,[r12]
    mov r3,#2
    mov r4,#1
    mov r5,#1
loop:
    add r6,r4,r5
    mov r4,r5
    mov r5,r6
    add r3,r3,#1
    add r7,r3,r3
    add r0,r7,r7
    add r0,r0,r11
    str r6,[r0]
    cmp r2,r3
    beq stop
    b loop
stop:
    b stop

.org 0x100
num:
    .word 10
.org 0x200
result:
    .word 0 0 0 0 0 0 0 0 0 0
```

第一步：打开源程序（fib\_armv7.s）

```
1 # 计算斐波那契数列          fib_armv7.s
2 # n=10 可以改变, 保存在0x100存储单元中
3 # 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、55.....)      0、1、1、2、3、5、8、0dh、
4 # 保存在0x200、0x204、0x208、...的存储单元中
5
6 .global _start
7 _start:
8 .org 0
9     ldr r10,=num
10    ldr r11,=result
11    ldr r2,[r10]
12    mov r1,#0
13    add r12,r11,#0
14    str r1,[r12]
15    mov r1,#1
16    add r12,r11,#4
17    str r1,[r12]
18    add r12,r11,#8
19    str r1,[r12]
20    mov r3,#2
21    mov r4,#1
22    mov r5,#1
23 loop:
24    add r6,r4,r5
25    mov r4,r5
26    mov r5,r6
27    add r3,r3,#1
28    add r7,r3,r3
29    add r0,r7,r7
30    add r0,r0,r11
31    str r6,[r0]
32    cmp r2,r3
33
```

第二步：汇编

```
Messages
Compiling...
Code and data loaded from ELF executable into memory. Total size is 568 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asm70aj8m.s.o work/asm70aj8m.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asm70aj8m.s.elf work/asm70aj8m.s.o
Compile succeeded.
```

第三步：连续执行程序

| Disassembly (Ctrl-D)                                                                          |          |                           |
|-----------------------------------------------------------------------------------------------|----------|---------------------------|
| Go to address, label, or register: <input type="text" value="00000000"/> <span>Refresh</span> |          |                           |
| Address                                                                                       | Opcode   | Disassembly               |
| 000001e4                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001e8                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001ec                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001f0                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001f4                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001f8                                                                                      | 00000000 | andeq r0, r0, r0          |
| 000001fc                                                                                      | 00000000 | andeq r0, r0, r0          |
|                                                                                               |          | result:                   |
| 00000200                                                                                      | 00000000 | andeq r0, r0, r0          |
| 00000204                                                                                      | 00000001 | andeq r0, r0, r1          |
| 00000208                                                                                      | 00000001 | andeq r0, r0, r1          |
| 0000020c                                                                                      | 00000002 | andeq r0, r0, r2          |
| 00000210                                                                                      | 00000003 | andeq r0, r0, r3          |
| 00000214                                                                                      | 00000005 | andeq r0, r0, r5          |
| 00000218                                                                                      | 00000008 | andeq r0, r0, r8          |
| 0000021c                                                                                      | 0000000d | andeq r0, r0, sp          |
| 00000220                                                                                      | 00000015 | andeq r0, r0, r5, LSL r0  |
| 00000224                                                                                      | 00000022 | andeq r0, r0, r2, LSR #32 |
| 00000228                                                                                      | 00000037 | andeq r0, r0, r7, LSR r0  |
|                                                                                               |          | 9 ldr r10,=num            |
| 0000022c                                                                                      | 00000100 | andeq r0, r0, r0, LSL #2  |
|                                                                                               |          | 10 ldr r11,=result        |
| 00000230                                                                                      | 00000200 | andeq r0, r0, r0, LSL #4  |
| 00000234                                                                                      | 00000000 | andeq r0, r0, r0          |
|                                                                                               |          | _end:                     |
| 00000238                                                                                      | aaaaaaaa | bge 0xfeaaace8            |
| 0000023c                                                                                      | aaaaaaaa | bge 0xfeaaacec            |
| 00000240                                                                                      | aaaaaaaa | bge 0xfeaaacf0            |
| 00000244                                                                                      | aaaaaaaa | bge 0xfeaaacf4            |
| 00000248                                                                                      | aaaaaaaa | bge 0xfeaaacf8            |
| 0000024c                                                                                      | aaaaaaaa | bge 0xfeaaacfc            |
| 00000250                                                                                      | aaaaaaaa | bge 0xfeaad00             |



| Go to address, label, or register: |                           |          |          |          | Refresh         |
|------------------------------------|---------------------------|----------|----------|----------|-----------------|
| Address                            | Memory contents and ASCII |          |          |          |                 |
| 000001f0                           | 00000000                  | 00000000 | 00000000 | 00000000 | ....            |
| 00000200                           | 00000000                  | 00000001 | 00000001 | 00000002 | ....            |
| 00000210                           | 00000003                  | 00000005 | 00000008 | 0000000d | ....            |
| 00000220                           | 00000015                  | 00000022 | 00000037 | 00000100 | ... " ... 7 ... |
| 00000230                           | 00000200                  | 00000000 | aaaaaaaa | aaaaaaaa | ....            |
| 00000240                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000250                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000260                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000270                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000280                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000290                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002a0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002b0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002c0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002d0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002e0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000002f0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000300                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000310                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000320                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000330                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000340                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000350                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000360                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000370                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000380                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 00000390                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000003a0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000003b0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000003c0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000003d0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |
| 000003e0                           | aaaaaaaa                  | aaaaaaaa | aaaaaaaa | aaaaaaaa | ....            |

## 回答问题:

(1) Helloworld\_mips.asm 源程序中，前后两个 syscall 语句的各自作用是什么？

```
main:
    li $v0,4
    la $a0,out_string
    syscall

    li $v0,10
    syscall
```

答：第一个 syscall 触发 MARS 系统调用，执行字符串打印，在控制台输出“Hello, World!”。第二个 syscall 触发 MARS 系统调用，结束程序运行，安全返回到操作系统，不再继续执行后续指令。

(2) 修改 sum\_mips.asm 源代码,设 n=100, 修改后程序命名为 **sum100\_mips.asm**, 编译运行。要求展示所使用寄存器和内存的存储情况。若将其机器码导出，如何操作？ 导出文件命名为 **sum100\_mips.hex**。

答：将代码第 5 行 10 改成 100 即可，运行后寄存器如图，其余见（1）e.。

| Registers |        |            | Coproc 1 | Coproc 0 |
|-----------|--------|------------|----------|----------|
| Name      | Number | Value      |          |          |
| \$zero    | 0      | 0x00000000 |          |          |
| \$at      | 1      | 0x00000000 |          |          |
| \$v0      | 2      | 0x0000000a |          |          |
| \$v1      | 3      | 0x00000000 |          |          |
| \$a0      | 4      | 0x00000000 |          |          |
| \$a1      | 5      | 0x00000000 |          |          |
| \$a2      | 6      | 0x00000000 |          |          |
| \$a3      | 7      | 0x00000000 |          |          |
| \$t0      | 8      | 0x00000000 |          |          |
| \$t1      | 9      | 0x00000000 |          |          |
| \$t2      | 10     | 0x00000000 |          |          |
| \$t3      | 11     | 0x00000000 |          |          |
| \$t4      | 12     | 0x00000000 |          |          |
| \$t5      | 13     | 0x00000000 |          |          |
| \$t6      | 14     | 0x00000000 |          |          |
| \$t7      | 15     | 0x00000000 |          |          |
| \$s0      | 16     | 0x00000064 |          |          |
| \$s1      | 17     | 0x00000064 |          |          |
| \$s2      | 18     | 0x00000001 |          |          |
| \$s3      | 19     | 0x000013ba |          |          |
| \$s4      | 20     | 0x00000000 |          |          |
| \$s5      | 21     | 0x00000000 |          |          |
| \$s6      | 22     | 0x00000000 |          |          |
| \$s7      | 23     | 0x00000000 |          |          |
| \$t8      | 24     | 0x00000000 |          |          |
| \$t9      | 25     | 0x00000000 |          |          |
| \$k0      | 26     | 0x00000000 |          |          |
| \$k1      | 27     | 0x00000000 |          |          |
| \$gp      | 28     | 0x00001800 |          |          |
| \$sp      | 29     | 0x00002ffc |          |          |
| \$fp      | 30     | 0x00000000 |          |          |
| \$ra      | 31     | 0x00000000 |          |          |
| pc        |        | 0x0000302c |          |          |
| hi        |        | 0x00000000 |          |          |
| lo        |        | 0x00000000 |          |          |

(3) 在 Logisim 实现的单周期 MIPS 处理器上，运行计算  $1+2+3+\dots+100$  的机器语言程序 (**sum100\_mips.hex**)，观看数据存储器 0 号单元的值是不是 13bah？程序运行的总周期数、无条件分支数、条件分支成功数分别是多少？

答：是，总周期数=404，无条件分支数=99，条件分支成功数=1。具体截图见 (2) c。

(4) 比较并分析 **sum\_riscv.asm** 与 **sum\_mips.asm** 两段代码在程序结束部分是如何处理的？

答：sum\_riscv.asm 使用 `jal zero, end` 使跳转到自己进入死循环，属于裸机环境下的硬件停机方式，sum\_mips.asm 使用 `addi $v0, $zero, 10 syscall` 系统调用使程序正常退出，属于操作系统环境下的软件退出方式。

(5) 修改 `sum_riscv.asm` 程序，计算  $1+2+3+\dots+100$ （修改后的程序命名为：**`sum100_riscv.asm`**），然后在 RARS 1.5 汇编仿真器中运行，并将该程序的机器码保存到 `sum100_riscv.hex` 文件中。

见（3）d。

(6) 针对在 X86 ISA 架构下，提供的 5 个 `helloworld.asm` 程序段，分析比较它们在语法、调用关系，展示等环节有何不同？

答：

|          | hello1                                 | hello2                                                  | hello3                                                               | hello4                                                                    | hello5                                             |
|----------|----------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------|
| 语法       | 直接使用<br>MASM<br>32 宏                   | 开启<br>__UNICODE_<br>_，使用过程定义<br>main proc,<br>main endp | 包含头文件<br>user32,gdi32,kernel<br>32，使用 chr\$(“字<br>符串”,13,10)处理换<br>行 | 有数据<br>段.data，<br>字符串定<br>义在数据<br>区，使用<br>invoke 调<br>用<br>Windows<br>API | 包含 C 标<br>准库<br>msvcrt.inc<br>，有数据<br>段.data       |
| 调用<br>关系 | print<br>“字符<br>串”<br>exit             | call main, cls,<br>inkey, print,<br>exit                | 宏 print，宏 exit，<br>依赖系统库                                             | invoke<br>MessageB<br>ox, invoke<br>ExitProces<br>s                       | invoke<br>crt_printf,<br>invoke<br>ExitProces<br>s |
| 展示       | 控制台<br>直接输<br>出字符<br>串，输<br>出后直<br>接退出 | 控制台输出，<br>输出后等待任<br>意键按下后退<br>出                         | 控制台输出，带换<br>行符格式化输出                                                  | 弹出图形<br>窗口                                                                | 控制台输<br>出，模拟<br>C 语言输<br>出风格                       |

(7) 对于 `sum_x86.asm` 源程序，其中 `print` 语句的功能是怎样的？如果在 8086ISA 架构下，其可替换为什么语句？请在编译器中验证，并给出相应的截屏佐证。

答： `print` 是宏指令，将字符串、数字等内容输出到控制台窗口，自动调用 Windows 系统底层 API 完成显示，这里输出字符串 “Sum=” 和变量 `sum` 的十进制数值。替换成 `int 21h` 的 `09h` 功能。

使用 dosbox 进行验证

```
x86.asm | fib_x86.asm | hello3_x86.asm | hello4_x86.asm | hello5_x86.asm | sum.asm
文件 编辑 查看

DATA SEGMENT
msg1 DB 'Input n= $' ; 输入提示
msg2 DB 13,10,'Sum= $' ; 输出提示
n DW ? ; 输入的n
sum DW ? ; 累加和
buf DB 10 DUP(?) ; 数字转字符串缓冲区
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START:
MOV AX,DATA
MOV DS,AX

; -----
; 输出 Input n=
; -----
LEA DX,msg1
MOV AH,09H
INT 21H

; -----
; 输入数字
; -----
MOV CX,0
MOV BX,0
INPUT_LOOP:
MOV AH,01H
INT 21H
CMP AL,0DH ; 回车结束
JE INPUT_END
SUB AL,30H ; 转数字
MOV CL,AL
MOV AX,BX
MOV BX,10
MUL BX
ADD AX,CX
MOV BX,AX
JMP INPUT_LOOP
INPUT_END:
MOV n,BX ; 正确保存n

; -----
; 累加 1+2+...+n
; -----
MOV CX,n
MOV AX,0
MOV BX,1
LOOP_SUM:
ADD AX,BX
INC BX
LOOP LOOP_SUM

MOV sum,AX ; 保存正确结果

; -----
; 输出 Sum=
; -----
LEA DX,msg2
MOV AH,09H
INT 21H

; -----
; 输出结果
; -----
MOV AX,sum
LEA DI,buf
MOV CX,0
MOV BX,10
NEXT_DIGIT:
XOR DX,DX
DIV BX
ADD DL,30H
PUSH DX
INC CX
CMP AX,0
JNE NEXT_DIGIT

PRINT_DIGIT:
POP DX
MOV AH,02H
INT 21H
LOOP PRINT_DIGIT

; -----
; 退出
; -----
MOV AH,4CH
INT 21H

CODE ENDS
CODE ENDS
```

```

DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
Object filename [sum_x86.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

51746 + 464798 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>link sum_x86.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [SUM_X86.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
LINK : warning L4021: no stack segment

C:\>sum_x86.exe
Input n= 10

Sum= 55
C:\>

```

(8) 比较下 `fib_armv7.s` 与 `fib_riscv.asm`、`fib_mips.asm` 是如何完成输入数据和结果的存储的？

输入：n

输出：0、1、1、2、3、5、8、13、21、34、...保存在地址为 0、4、8、12、...的数据存储器中

答：

|      | <code>fib_armv7.s</code>                                                       | <code>fib_mips.asm</code>                            | <code>fib_riscv.asm</code>                       |
|------|--------------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------------|
| 输入数据 | 存放在固定地址 0x100，用 <code>.org 0x100</code> 进行定位，用 <code>ldr r10,=num</code> 取地址读取 | 直接用立即数写在指令里，用 <code>addi \$s2,\$zero,10</code> 存入寄存器 | 直接用立即数存入寄存器，使用指令 <code>ori a2,zero,10</code>     |
| 结果   | 从固定地址 0x200 开始存放                                                               | 从地址 0x0000_0000 开始连续存放，使用 <code>\$zero</code> 基址寻址   | 从地址 0x0000_0000 开始连续存放，使用 <code>zero</code> 作为基址 |

(9) `fib_armv7.s` 中为何有数据段 `.word`，

```

.org 0x100
num:
    .word 10
.org 0x200
result:
    .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

```

是否能改成与其他两套指令代码一样的形式？

答: fib\_armv7.s 使用了固定内存布局, 输入放在 0x100, 结果放在 0x200, 所以用数据段.word 来指定地址开辟存储单元。如果要改成与其他两套指令代码一样的形式就必须舍去使用固定内存布局, 改为使用立即数表示 n, 结果从地址 0 开始存储。

### 3.2 设计实验

(1) MIPS 汇编语言程序设计(排序程序)——文件名: sort1\_mips.asm (降序排序)、sort2\_mips.asm (升序排序)。

假设有 10 个数据, 这 10 个数据的顺序是随机的, 例如: 8、1、5、2、7、9、6、4、3、10; 这 10 个数据是通过指令存放在数据存储器的第 0、4、8、...、36 号地址的存储单元中。

a. 对这 10 个数据按照从大到小的顺序(降序)进行排序, 排序后的数据仍然放在数据存储器的第 0、4、8、...、36 号地址的存储单元中。

编写 sort1\_mips.asm 程序



```
ASM sort1_mips.asm X ASM sort2_mips.asm
mips > ASM sort1_mips.asm
2   main:
3       lw $s0,$zero,0
4       sw $s0,0($zero)
5       addi $s0,$zero,1
6       sw $s0,4($zero)
7       addi $s0,$zero,5
8       sw $s0,8($zero)
9       addi $s0,$zero,2
10      sw $s0,12($zero)
11      addi $s0,$zero,7
12      sw $s0,16($zero)
13      addi $s0,$zero,9
14      sw $s0,20($zero)
15      addi $s0,$zero,6
16      sw $s0,24($zero)
17      addi $s0,$zero,4
18      sw $s0,28($zero)
19      addi $s0,$zero,3
20      sw $s0,32($zero)
21      addi $s0,$zero,10
22      sw $s0,36($zero)
23
24
25      # 比较
26      addi $s1,$zero,0
27      addi $s2,$zero,0
28      addi $t0,$zero,9
29  # 外层循环
30  loop1:
31      addi $s2,$zero,0
32  # 内层循环
33  loop2:
34      addi $s0,$s2,0
35      sll $s5,$s0,2
36      lw $s3,0($s5)
37      addi $s0,$s2,1
38      sll $s6,$s0,2
39      lw $s4,0($s6)
40      # 小于则交换
41      slt $s7,$s3,$s4
42      beq $s7,$zero,skip
43      sw $s4,0($s5)
44      sw $s3,0($s6)
45  skip:
46      addi $s2,$s2,1
47      sub $s0,$t0,$s1
48      bne $s2,$s0,loop2
49      addi $s1,$s1,1
50      sub $s0,$t0,$s1
51      bne $s0,$zero,loop1
52  finish:
53      addi $v0,$zero,10
```

在 MIPS 汇编器上运行通过

| Data Segment |            |            |            |            |             |             |             |             |
|--------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| Address      | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
| 0x00000000   | 0x0000000a | 0x00000009 | 0x00000008 | 0x00000007 | 0x00000006  | 0x00000005  | 0x00000004  | 0x00000003  |
| 0x00000020   | 0x00000002 | 0x00000001 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000040   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000060   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000080   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

导出汇编后的机器码并增加 v2.0 raw

```
ASM sort2_mips.asm  ASM sort1_mips.asm  ≡ sort1_mips.hex X
mips > ≡ sort1_mips.hex
1  v2.0 raw
2  20100008
3  ac100000
4  20100001
5  ac100004
6  20100005
7  ac100008
8  20100002
9  ac10000c
10 20100007
11 ac100010
12 20100009
13 ac100014
14 20100006
15 ac100018
16 20100004
17 ac10001c
18 20100003
19 ac100020
20 2010000a
21 ac100024
22 20110000
23 20120000
24 20080009
25 20120000
26 22500000
27 0010a880
28 8eb30000
29 22500001
30 0010b080
31 8ed40000
32 0274b82a
33 12e00002
34 aeb40000
35 aed30000
36 22520001
37 01118022
38 1650fff3
39 22310001
40 01118022
41 1600ffef
42 2002000a
43 0000000c
44
```

在单周期 MIPS 处理器数据通路的指令存储器中装入机器码文件 sort1\_mips.hex

| 工程  | 电路仿真     | 窗口       | 帮助       |          |          |          |          |          |          |          |          |          |          |          |          |          |  |  |  |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|--|--|--|
| 000 | 20100000 | ac100000 | 20100001 | ac100004 | 20100005 | ac100008 | 20100002 | ac10000c | 20100007 | ac100010 | 20100009 | ac100014 | 20100006 | ac100018 | 20100004 | ac10001c |  |  |  |
| 010 | 20100003 | ac100020 | 2010000a | ac100024 | 20110000 | 20120000 | 20080009 | 20120000 | 22500000 | 0010a880 | 8eb30000 | 22500001 | 0010b080 | 8ed40000 | 0274b82a | 12a00002 |  |  |  |
| 020 | aeb40000 | aed30000 | 22520001 | 01118022 | 1650fff3 | 22310001 | 01118022 | 1600ffef | 2002000a | 0000000c | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |  |  |  |
| 030 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |  |  |  |
| 040 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |  |  |  |
| 050 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |  |  |  |

运行后数据存储器

| 工程  | 电路仿真     | 窗口       | 帮助       |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000 | 00000000 | 00000009 | 00000007 | 00000006 | 00000005 | 00000004 | 00000003 | 00000002 | 00000001 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 010 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 030 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

b. 对这 10 个数据按照从小到大的顺序（升序）进行排序，排序后的数据仍然放在数据存储器的第 0、4、 8、 ...、 36 号地址的存储单元中。

编写 sort2\_mips.asm 程序

sort1\_mips.asm

sort2\_mips.asm X

mips > sort2\_mips.asm

```

2   main:
3       # 初始化数组
4       sw $s0,0($zero)
5       sw $s0,0($zero)
6       addi $s0,$zero,1
7       sw $s0,4($zero)
8       addi $s0,$zero,5
9       sw $s0,8($zero)
10      addi $s0,$zero,2
11      sw $s0,12($zero)
12      addi $s0,$zero,7
13      sw $s0,16($zero)
14      addi $s0,$zero,9
15      sw $s0,20($zero)
16      addi $s0,$zero,6
17      sw $s0,24($zero)
18      addi $s0,$zero,4
19      sw $s0,28($zero)
20      addi $s0,$zero,3
21      sw $s0,32($zero)
22      addi $s0,$zero,10
23      sw $s0,36($zero)
24
25      # 比较
26      addi $s1,$zero,0
27      addi $s2,$zero,0
28      addi $t0,$zero,9
29      # 外层循环
30      loop1:
31          addi $s2,$zero,0
32      # 内层循环
33      loop2:
34          addi $s0,$s2,0
35          sll $s5,$s0,2
36          lw $s3,0($s5)
37          addi $s0,$s2,1
38          sll $s6,$s0,2
39          lw $s4,0($s6)
40          # 大于则交换
41          sgt $s7,$s3,$s4
42          beq $s7,$zero,skip
43          sw $s4,0($s5)
44          sw $s3,0($s6)
45      skip:
46          addi $s2,$s2,1
47          sub $s0,$t0,$s1
48          bne $s2,$s0,loop2
49          addi $s1,$s1,1
50          sub $s0,$t0,$s1
51          bne $s0,$zero,loop1
52      finish:
53          addi $v0,$zero,10

```

在 MIPS 汇编器上运行通过

| Data Segment |            |            |            |            |             |             |             |             |
|--------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| Address      | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
| 0x00000000   | 0x00000001 | 0x00000002 | 0x00000003 | 0x00000004 | 0x00000005  | 0x00000006  | 0x00000007  | 0x00000008  |
| 0x00000020   | 0x00000009 | 0x0000000a | 0x0000000b | 0x0000000c | 0x0000000d  | 0x0000000e  | 0x0000000f  | 0x00000010  |
| 0x00000040   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000060   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000080   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x000000a0   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

导出汇编后的机器码并增加 v2.0 raw



ASM sort2\_mips.asm

ASM sort1\_mips.asm

sort1\_mips.hex

sort2\_mips.hex X

mips > sort2\_mips.hex

1

v2.0 raw

2

20100008

3

ac100000

4

20100001

5

ac100004

6

20100005

7

ac100008

8

20100002

9

ac10000c

10

20100007

11

ac100010

12

20100009

13

ac100014

14

20100006

15

ac100018

16

20100004

17

ac10001c

18

20100003

19

ac100020

20

2010000a

21

ac100024

22

20110000

23

20120000

24

20080009

25

20120000

26

22500000

27

0010a880

28

8eb30000

29

22500001

30

0010b080

31

8ed40000

32

0293b82a

33

12e00002

34

aeb40000

35

aed30000

36

22520001

37

01118022

38

1650fff3

39

22310001

40

01118022

41

1600ffeef

42

2002000a

43

0000000c

44

在单周期 MIPS 处理器数据通路的指令存储器中装入机器码文件 sort2\_mips.hex

| 电路仿真 窗口 帮助 |          |          |          |          |          |          |          |           |          |          |          |          |          |          |          |          |
|------------|----------|----------|----------|----------|----------|----------|----------|-----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000        | 20100008 | ac100000 | 20100001 | ac100004 | 20100005 | ac100008 | 20100002 | ac10000c  | 20100007 | ac100010 | 20100009 | ac100014 | 20100006 | ac100018 | 20100004 | ac10001c |
| 010        | 20100003 | ac100020 | 2010000a | ac100024 | 20110000 | 20120000 | 20080009 | 20120000  | 22500000 | 0010a880 | 8eb30000 | 22500001 | 0010b080 | 8ed40000 | 0293b82a | 12e00002 |
| 020        | aeb40000 | aed30000 | 22520001 | 01118022 | 1650fff3 | 22310001 | 01118022 | 1600ffeef | 2002000a | 0000000c | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 030        | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000  | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

运行后数据存储器

|     |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000 | 00000000 | 00000001 | 00000002 | 00000003 | 00000004 | 00000005 | 00000006 | 00000007 | 00000009 | 0000000a | 00000000 | 00000000 |
| 010 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 030 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 040 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

(2) RISC-V 汇编语言程序设计（排序程序）——文件名：sort1\_riscv.asm（降序排序）、sort2\_riscv.asm（升序排序）。

假设有 10 个数据，这 10 个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10；这 10 个数据是通过指令存放在数据存储器的第 0、4、8、…、36 号地址的存储单元中。

a. 对这 10 个数据按照从大到小的顺序（降序）进行排序，排序后的数据仍然放在数据存储器的第 0、4、8、…、36 号地址的存储单元中。

编写 sort1\_riscv.asm 程序

```
sort2_mips.asm  sort1_mips.asm  sort1_mips.hex  sort2_mips.hex  sort1_riscv.asm X
riscv > sort1_riscv.asm
1  main:
2      # 存储数据
3      ori s0,zero,10
4      sw s0,0(zero)
5      add t0,s0,zero
6      ori s0,zero,-1
7      add t0,t0,s0
8
9      ori s0,zero,8
10     sw s0,4(zero)
11     ori s0,zero,1
12     sw s0,8(zero)
13     ori s0,zero,5
14     sw s0,12(zero)
15     ori s0,zero,2
16     sw s0,16(zero)
17     ori s0,zero,7
18     sw s0,20(zero)
19     ori s0,zero,9
20     sw s0,24(zero)
21     ori s0,zero,6
22     sw s0,28(zero)
23     ori s0,zero,4
24     sw s0,32(zero)
25     ori s0,zero,3
26     sw s0,36(zero)
27     ori s0,zero,10
28     sw s0,40(zero)
29
30     # 比较
31     ori s1,zero,0
32     ori s2,zero,0
33     ori s9,zero,1
34     ori s10,zero,4
35 # 外层循环
36 loop1:
37     ori s8,zero,4
38     ori s2,zero,0
39 # 内层循环
40 loop2:
41     add s5,s8,zero
42     lw s3,0(s5)
43     add s8,s8,s10
44     add s6,s8,zero
45     lw s4,0(s6)
46     slt s7,s4,s3
47     beq s7,zero,go
48     beq zero,zero,skip
49 go:
50     sw s4,0(s5)
```

```

51      sw s3,0(s6)
52  skip:
53      add s2,s2,s9
54      add s0,s1,s2
55      slt s7,t0,s0
56      beq s7,zero,loop2
57      add s1,s1,s9
58      slt s7,t0,s1
59      beq s7,zero,loop1
60  end:
61      jal zero,end

```

在 RISC-V 汇编器上运行通过

| Data Segment |            |            |            |            |             |             |             |             |
|--------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| Address      | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
| 0x00000000   | 0x0000000a | 0x0000000a | 0x00000009 | 0x00000008 | 0x00000007  | 0x00000006  | 0x00000005  | 0x00000004  |
| 0x00000020   | 0x00000003 | 0x00000002 | 0x00000001 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000040   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000060   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

导出汇编后的机器码并增加 v2.0 raw

```

sort2_mips.asm  sort1_mips.asm  sort1_mips.hex  sort2_mips.hex  sort1_riscv.asm  sort2_riscv.asm  sort1_riscv.hex
riscv > sort1_riscv.hex
1  v2.0 raw
2  00a06413
3  00802023
4  000402b3
5  fff06413
6  008282b3
7  00806413
8  00802223
9  00106413
10 00802423
11 00506413
12 00802623
13 00206413
14 00802823
15 00706413
16 00802a23
17 00906413
18 00802c23
19 00606413
20 00802e23
21 00406413
22 02802023
23 00306413
24 02802223
25 00a06413
26 02802423
27 00006493
28 00006913
29 00106c93
30 00406d13
31 00406c13
32 00006913
33 000c0ab3
34 000aa983
35 01ac0c33
36 000c0b33
37 000b2a03
38 013a2bb3
39 000b8463
40 00000663
41 014aa023
42 013b2023
43 01990933
44 01248433
45 0082abb3
46 fc0b86e3
47 019484b3
48 0092abb3
49 fa0b8ce3
50 0000006f

```

在单周期 RISC-V 处理器数据通路的指令存储器中装入机器码文件 sort1\_riscv.hex

电路仿真 窗口 帮助

|     |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000 | 00a06413 | 00802023 | 000402b3 | fff06413 | 008282b3 | 00806413 | 00802223 | 00106413 | 00802423 | 00506413 | 00802623 | 00206413 | 00802823 | 00706413 | 00802a23 | 00906413 |
| 010 | 00802c23 | 00606413 | 00802e23 | 00406413 | 02802023 | 00306413 | 02802223 | 00a06413 | 02802423 | 00006493 | 00006913 | 00106c93 | 00406d13 | 00406c13 | 00006913 | 000c0ab3 |
| 020 | 000aa983 | 01ac0c33 | 000c0b33 | 000b2a03 | 013a2bb3 | 000b8463 | 00000663 | 014aa023 | 013b2023 | 01990933 | 01248433 | 0082abb3 | fc0b86e3 | 019484b3 | 0092abb3 | fa0b8ce3 |
| 030 | 0000006f | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 040 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 050 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

运行后数据存储器

|        |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|--------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 000000 | 0000000a | 0000000a | 00000009 | 00000008 | 00000007 | 00000006 | 00000005 | 00000004 | 00000003 | 00000002 | 00000001 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000010 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |
| 000020 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 | 00000000 |

b. 对这 10 个数据按照从小到大的顺序（升序）进行排序，排序后的数据仍然放在数据存储器的第 0、4、8、…、36 号地址的存储单元中。

编写 sort2\_riscv.asm 程序

```

ASM sort2_mips.asm  ASM sort1_mips.asm  sort1_mips.hex  sort2_mips.hex  sort1_riscv.asm  sort2_riscv.asm X
riscv > ASM sort2_riscv.asm
1  main:
2      # 存储数据
3      ori s0,zero,10
4      sw s0,0(zero)
5      add t0,s0,zero
6      ori s0,zero,-1
7      add t0,t0,s0
8
9      ori s0,zero,8
10     sw s0,4(zero)
11     ori s0,zero,1
12     sw s0,8(zero)
13     ori s0,zero,5
14     sw s0,12(zero)
15     ori s0,zero,2
16     sw s0,16(zero)
17     ori s0,zero,7
18     sw s0,20(zero)
19     ori s0,zero,9
20     sw s0,24(zero)
21     ori s0,zero,6
22     sw s0,28(zero)
23     ori s0,zero,4
24     sw s0,32(zero)
25     ori s0,zero,3
26     sw s0,36(zero)
27     ori s0,zero,10
28     sw s0,40(zero)
29
30     # 比较
31     ori s1,zero,0
32     ori s2,zero,0
33     ori s9,zero,1
34     ori s10,zero,4
35     # 外层循环
36     loop1:
37         ori s8,zero,4
38         ori s2,zero,0
39     # 内层循环
40     loop2:
41         add s5,s8,zero
42         lw s3,0(s5)
43         add s8,s8,s10
44         add s6,s8,zero
45         lw s4,0(s6)
46         slt s7,s4,s3
47         beq s7,zero,skip
48         sw s4,0(s5)
49         sw s3,0(s6)
50     skip:

```

```

51     add s2,s2,s9
52     add s0,s1,s2
53     slt s7,t0,s0
54     beq s7,zero,loop2
55     add s1,s1,s9
56     slt s7,t0,s1
57     beq s7,zero,loop1
58 end:
59     jal zero,end

```

在 RISC-V 汇编器上运行通过



| Data Segment |            |            |            |            |             |             |             |             |
|--------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| Address      | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
| 0x00000000   | 0x0000000a | 0x00000000 | 0x00000001 | 0x00000002 | 0x00000003  | 0x00000004  | 0x00000005  | 0x00000006  |
| 0x00000020   | 0x00000007 | 0x00000008 | 0x00000009 | 0x0000000a | 0x0000000b  | 0x0000000c  | 0x0000000d  | 0x0000000e  |
| 0x00000040   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00000060   | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

导出汇编后的机器码并增加 v2.0 raw

```
sort1_mips.asm  sort1_mips.hex  sort2_mips.hex  sort1_riscv.asm  sort2_riscv.asm  sort1_riscv.hex  sort2_riscv.hex X
riscv > sort2_riscv.hex
1  v2.0 raw
2  00a06413
3  00802023
4  00402b3
5  fff06413
6  008282b3
7  00806413
8  00802223
9  00106413
10 00802423
11 00506413
12 00802623
13 00206413
14 00802823
15 00706413
16 00802a23
17 00906413
18 00802c23
19 00606413
20 00802e23
21 00406413
22 02802023
23 00306413
24 02802223
25 00a06413
26 02802423
27 0006493
28 0006913
29 00106c93
30 00406d13
31 00406c13
32 0006913
33 000c0ab3
34 000aa983
35 01ac0c33
36 000c0b33
37 000b2a03
38 013a2bb3
39 000b8663
40 014aa023
41 013b2023
42 01990933
43 01248433
44 0082abb3
45 fc0b88e3
46 019484b3
47 0092abb3
48 fa0b8ee3
49 0000006f
```

在单周期 RISC-V 处理器数据通路的指令存储器中装入机器码文件 sort2\_riscv.hex

| 电路仿真 |                                                                                                                                                 | 窗口 | 帮助 |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------|----|----|
| 000  | 00a06413 00802023 00402b3 fff06413 008282b3 00806413 00802223 00106413 00802423 00506413 00802623 00206413 00802823 00706413 00802a23 00906413  |    |    |
| 010  | 00802c23 00606413 00802e23 00406413 02802023 00306413 02802223 00a06413 02802423 0006493 0006913 00106c93 00406d13 00406c13 0006913 000c0ab3    |    |    |
| 020  | 000aa983 01ac0c33 000c0b33 000b2a03 013a2bb3 000b8663 014aa023 013b2023 01990933 01248433 0082abb3 fc0b88e3 019484b3 0092abb3 fa0b8ee3 0000006f |    |    |
| 030  | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000          |    |    |
| 040  | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000          |    |    |
| 050  | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000          |    |    |

运行后数据存储器

| 电路仿真   |                                                                                                                                        | 窗口 | 帮助 |
|--------|----------------------------------------------------------------------------------------------------------------------------------------|----|----|
| 000000 | 0000000a 00000000 00000001 00000002 00000003 00000004 00000005 00000006 00000007 00000008 00000009 0000000a 00000000 00000000 00000000 |    |    |
| 000010 | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 |    |    |
| 000020 | 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 |    |    |

(3) Intel x86 汇编语言程序设计（排序程序）——文件名：sort1\_x86.asm（降序排序）、sort2\_x86.asm（升序排序）。

假设有 10 个数据，这 10 个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10。

a. 对这 10 个数据按照从大到小的顺序（降序）进行排序。

编写 sort1\_x86.asm

```
sort2_mips.hex  ASM sort1_riscv.asm  ASM sort2_riscv.asm  sort1_riscv.hex  sort2_riscv.hex  ASM sort1_x86.asm X  ASM /
x86 > ASM sort1_x86.asm
1  .486
2  .model flat, stdcall
3  option casemap :none
4  include \masm32\include\windows.inc
5  include \masm32\include\user32.inc
6  include \masm32\include\kernel32.inc
7  includelib \masm32\lib\user32.lib
8  includelib \masm32\lib\kernel32.lib
9
10 .data
11     array db 8,1,5,2,7,9,6,4,3,10
12     count equ 10
13     newline db 13,10,0
14     space db ' ',0
15     buffer db 5 dup(0)
16 .code
17
18 start:
19     mov ecx,count-1
20     mov esi,0
21 outer_loop:
22     mov edi,0
23 inner_loop:
24     mov al,[array+edi]
25     mov bl,[array+edi+1]
26     cmp al,bl
27     jge no_swap
28
29     mov [array+edi],bl
30     mov [array+edi+1],al
31 no_swap:
32     inc edi
33     cmp edi,ecx
34     jl inner_loop
35     dec ecx
36     cmp ecx,0
37     jg outer_loop
38
39     mov esi,0
40 print_loop:
41     movzx eax, byte ptr [array+esi]
42     mov edi,offset buffer+4
43     mov byte ptr [edi],0
44     mov ebx,10
45
46 convert_loop:
47     dec edi
48     xor edx,edx
49     div ebx
50     add dl,'0'
```

```

51     mov [edi],dl
52     test eax,eax
53     jnz convert_loop
54
55     push offset buffer
56     call GetStdHandle
57     push STD_OUTPUT_HANDLE
58     call GetStdHandle
59     push 0
60     push offset bytesWritten
61     push 5
62     push edi
63     push eax
64     call WriteFile
65
66     inc esi
67     cmp esi, count
68     jge no_space
69     push offset space
70     call GetStdHandle
71     push STD_OUTPUT_HANDLE
72     call GetStdHandle
73     push 0
74     push offset bytesWritten
75     push 1
76     push offset space
77     push eax
78     call WriteFile
79 no_space:
80     cmp esi, count
81     jl print_loop
82
83     push offset newline
84     call GetStdHandle
85     push STD_OUTPUT_HANDLE
86     call GetStdHandle
87     push 0
88     push offset bytesWritten
89     push 2
90     push offset newline
91     push eax
92     call WriteFile
93
94     invoke ExitProcess,NULL
95 .data?
96     bytesWritten dd ?
97 end start

```

编译运行

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. 保留所有权利。

E:\lab4\x86>sort1_x86.exe
10 9 8 7 6 5 4 3 2 1

E:\lab4\x86>|
```

b. 对这 10 个数据按照从小到大的顺序（升序）进行排序。

编写 sort2\_x86.asm

```
22     mov edi,0
23     inner_loop:
24         mov al,[array+edi]
25         mov bl,[array+edi+1]
26         cmp al,bl
27         jle no_swap
28
```

在 sort1 的基础上，把 jge 改为 jle 即可

编译运行

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. 保留所有权利。

E:\lab4\x86>sort1_x86.exe
10 9 8 7 6 5 4 3 2 1

E:\lab4\x86>sort2_x86.exe
1 2 3 4 5 6 7 8 9 10

E:\lab4\x86>|
```

### 3.3 挑战性实验（完成可+6分）

ARMv7 汇编语言程序设计（排序程序）——文件名：sort1\_armv7.s（降序排序）、sort2\_armv7.s（升序排序）。

假设有 10 个数据，这 10 个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10。

a. 对这 10 个数据按照从大到小的顺序（降序）进行排序。

编写 sort1\_armv7.s

```

1  .global _start
2  _start:
3      ldr r0,=10
4      mov r1,#0
5  outer_loop:
6      add r1,r1,#1
7      cmp r1,r0
8      beq end_start
9      mov r2,#0
10     ldr r6,=arr
11     mov r3,#0
12  inner_loop:
13     ldr r4,[r6]
14     ldr r5,[r6,#4]
15
16     cmp r4,r5
17     bge no_swap
18     str r5,[r6]
19     str r4,[r6,#4]
20
21     mov r3,#1
22  no_swap:
23     add r6,r6,#4
24     add r2,r2,#1
25     sub r7,r0,r1
26     cmp r2,r7
27     blo inner_loop
28
29     cmp r3,#0
30     beq end_start
31     b outer_loop
32  end_start:
33     mov r7,#1
34     mov r0,#0
35     svc 0
36     .data
37     .align 2
38  arr:
39     .word 8,1,5,2,7,9,6,4,3,10

```

编译运行



|          |          |       |                    |
|----------|----------|-------|--------------------|
| 00000068 | 00000070 | andeq | r0, r0, r0, ROR r0 |
| 0000006c | 00000000 | andeq | r0, r0, r0         |
| arr:     |          |       |                    |
| 00000070 | 0000000a | andeq | r0, r0, r10        |
| 00000074 | 00000009 | andeq | r0, r0, r9         |
| 00000078 | 00000008 | andeq | r0, r0, r8         |
| 0000007c | 00000007 | andeq | r0, r0, r7         |
| 00000080 | 00000006 | andeq | r0, r0, r6         |
| 00000084 | 00000005 | andeq | r0, r0, r5         |
| 00000088 | 00000004 | andeq | r0, r0, r4         |
| 0000008c | 00000003 | andeq | r0, r0, r3         |
| 00000090 | 00000002 | andeq | r0, r0, r2         |
| 00000094 | 00000001 | andeq | r0, r0, r1         |

b. 对这 10 个数据按照从小到大的顺序（升序）进行排序。

编写 sort2\_armv7.s

```
sort1_armv7.s  sort2_armv7.s X
sort2_armv7.s
1  .global _start
2  _start:
3      ldr r0,=10
4      mov r1,#0
5  outer_loop:
6      add r1,r1,#1
7      cmp r1,r0
8      beq end_start
9      mov r2,#0
10     ldr r6,=arr
11     mov r3,#0
12  inner_loop:
13     ldr r4,[r6]
14     ldr r5,[r6,#4]
15
16     cmp r4,r5
17     ble no_swap
18     str r5,[r6]
19     str r4,[r6,#4]
20
21     mov r3,r3,#1
22  no_swap:
23     add r6,r6,#4
24     add r2,r2,#1
25     sub r7,r0,r1
26     cmp r2,r7
27     blo inner_loop
28
29     cmp r3,#0
30     beq end_start
31     b outer_loop
32  end_start:
33     mov r7,#1
34     mov r0,#0
35     svc 0
36     .data
37     .align 2
38  arr:
39     .word 8,1,5,2,7,9,6,4,3,10
```

编译运行

|          |          |       |            |
|----------|----------|-------|------------|
| 0000006c | 00000000 | andeq | r0, r0, r0 |
|          |          | arr:  |            |
| 00000070 | 00000001 | andeq | r0, r0, r1 |
| 00000074 | 00000002 | andeq | r0, r0, r2 |
| 00000078 | 00000003 | andeq | r0, r0, r3 |
| 0000007c | 00000004 | andeq | r0, r0, r4 |
| 00000080 | 00000005 | andeq | r0, r0, r5 |
| 00000084 | 00000006 | andeq | r0, r0, r6 |
| 00000088 | 00000007 | andeq | r0, r0, r7 |
| 0000008c | 00000008 | andeq | r0, r0, r8 |
| 00000090 | 00000009 | andeq | r0, r0, r9 |
| 00000094 | aaaaaaaa | bge   | 0xfeaab44  |